MSc Cyber Security, Threat Intelligence and Forensics

School of Science, Engineering and Environment

2024-2025

MSc Dissertation

CI/CD Pipeline Abuse in Cloud Deployments: From Code Injection to Infrastructure Takeover

Author:

Umesh Udyan

U.Udayan@edu.salford.ac.uk

@00792418

Supervisor:

Dr. Muhammad Ateeq

M.Ateeq@salford.ac.uk

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Muhammad Ateeq, for his invaluable guidance, feedback, and academic support throughout this dissertation. His insights and encouragement were instrumental in shaping both the direction and quality of this research.

I am deeply thankful to my parents and sister for their unwavering support, love, and patience throughout this journey. Their belief in me gave me the strength to persist through every challenge.

I also wish to extend my appreciation to the University of Salford, whose learning environment, resources, and dedicated academic staff provided the foundation for my growth. Special thanks to my classmates and course mentors for the collaborative spirit, meaningful discussions, and shared motivation that enriched my experience.

Finally, I would like to thank my close friends whose encouragement, honest conversations, and constant support made this journey more manageable and meaningful.

Abstract

Continuous Integration and Continuous Deployment (CI/CD) pipelines have become essential for modern software delivery, but their deep integration with cloud platforms has created new attack surfaces. Misconfigurations such as hardcoded secrets, unpinned GitHub Actions, and over-permissive IAM roles have been repeatedly exploited in high-profile breaches, enabling attackers to escalate from code injection to full infrastructure takeover. While cloud providers like AWS offer detection services including GuardDuty, CloudTrail, and IAM Access Analyzer, there has been limited empirical testing of how these tools perform under real CI/CD-originated compromises.

This dissertation addresses this gap by constructing a reproducible, sandboxed lab environment using GitHub Actions, Terraform, and AWS Free Tier. Three representative attack scenarios were simulated: pull request metadata injection, poisoned GitHub Actions, and IAM credential misuse. These scenarios were mapped to the MITRE ATT&CK framework and OWASP CI/CD Top 10, and evaluated using AWS-native detection services alongside open-source scanners such as TruffleHog, Gitleaks, and Terrascan.

Findings show that while CloudTrail reliably captured activity, GuardDuty produced alerts mainly for external or brute-force style misuse, failing to detect insider-style attacks conducted through valid CI/CD roles. IAM Access Analyzer flagged over-permissive policies only after deployment, highlighting its reactive nature. Overall, detection coverage was biased toward traditional credential theft, leaving blind spots for pipeline-based privilege escalation and supply chain abuse.

To address these weaknesses, the study applies CVSS v3.1 scoring to quantify the severity of each scenario and develops a five-pillar DevSecOps hardening framework. This framework integrates secrets management, least-privilege IAM design, action verification, policy-as-code enforcement, and runtime monitoring. The contribution is both academic and practical: it provides empirical evidence of detection gaps in AWS-native tooling and delivers a reproducible checklist for securing CI/CD pipelines in cloud environments.

Contents

Chapter 1 – Introduction

1.1 Background and Motivation

1.1.1 CI/CD Pipelines in Cloud Environments

Continuous Integration and Continuous Deployment (CI/CD) pipelines have become the backbone of modern software delivery. By automating builds, testing, and releases, CI/CD shortens delivery cycles and enables organisations to scale development across distributed teams. These pipelines are now tightly coupled with cloud platforms, where Infrastructure-as-Code (IaC) tools such as Terraform allow entire environments servers, storage, and networking to be provisioned automatically in minutes (Rahman, Williams, & Meneely, 2021).

While these changes have increased agility, they have also widened the attack surface. A single misconfigured template or insecure workflow can propagate instantly across environments, magnifying its impact. As organisations adopt DevSecOps, the resilience of CI/CD pipelines has **No table of figures entries found.**become inseparable from the overall security posture of the cloud (Fowler & Foote, 2021; Sharma, Kaushik, & Gupta, 2022).

1.1.2 The Security Challenge

High-profile incidents highlight the consequences of pipeline compromise. The SolarWinds Orion supply chain breach (2020) revealed how adversaries could insert malicious code during build processes, compromising thousands of downstream customers (Zhang, Chen, & Zhou, 2022). The Codecov Bash Uploader breach (2021) showed that pipeline scripts could be modified to exfiltrate secrets (Mitre, 2021). In parallel, dependency confusion attacks, highlighted by Birsan (2021), exploited package naming conventions to poison builds.

These cases underscore that pipelines are no longer peripheral concerns but primary attack targets. In recognition of this, the OWASP CI/CD Top 10 (2023) listed risks such as hardcoded secrets, unpinned third-party actions, and over-permissive access roles as systemic threats. Empirical research also highlights developer trade-offs, where speed and convenience often override secure practices, creating exploitable weaknesses (Beller, Gousios, Zaidman, & Juergens, 2022). Industry surveys add weight: Wiz (2023) found that nearly 77% of organisations had exposed secrets in production pipelines, while Sysdig (2024) reported that IaC misconfigurations accounted for more than half of critical cloud vulnerabilities.

1.1.3 Existing Tools and Their Limits

Cloud providers, including AWS, offer built-in security services intended to detect or mitigate misconfigurations. GuardDuty analyses telemetry for anomalies, CloudTrail provides an auditable record of API events, and IAM Access Analyzer reviews policies for excessive privileges. In practice, these services are widely deployed and form the default detection layer in many organisations (AWS, 2024).

Yet limitations remain. GuardDuty may detect brute-force logins but struggles to identify credential misuse when valid keys are used in pipelines (Palo Alto Unit 42, 2024). CloudTrail can provide comprehensive logging, but if trails are disabled or logs deleted, visibility is lost (Wiz, 2023). Even IAM Access Analyzer is reactive, flagging issues only after deployment. Open-source scanners such

as TruffleHog and Gitleaks add another layer by scanning repositories for embedded secrets, but their performance varies depending on configuration and regex rules (Sinha, Gupta, & Singh, 2022).

This highlights a central gap: there is limited empirical, reproducible testing of AWS-native monitoring tools against real CI/CD-originated threats. Addressing this gap forms the basis of this dissertation.

1.2 Research Objectives

The primary aim of this research is to investigate how insecure CI/CD pipelines can be exploited to compromise cloud environments and to evaluate the effectiveness of AWS-native monitoring tools. This aim is broken down into four objectives:

Table 1.1 – SMART Objectives of the Dissertation

Objective	Description	Expected Outcome
O1 – Lab Setup	Build a controlled CI/CD pipeline using GitHub Actions, Terraform, and AWS Free Tier.	Reproducible experimental environment with dummy data.
O2 – Attack Simulation	Simulate CI/CD attacks, including hardcoded secrets, poisoned actions, and over-permissive IAM policies.	Reproduction of real-world CI/CD-originated threats.
O3 – Detection Evaluation	Test GuardDuty, CloudTrail, IAM Access Analyzer, and scanners (TruffleHog, Gitleaks).	Empirical evidence of detection coverage and blind spots.
O4 – Risk Assessment & Framework	Apply CVSS v3.1 scoring and propose mitigation strategies.	Practical DevSecOps framework for pipeline resilience.

Together, these objectives address the research gap by providing reproducible evidence on how AWS-native monitoring tools respond to CI/CD-originated threats.

1.3 Research Approach

This study follows a Design Science Research (DSR) methodology, combined with an Agile-inspired iterative approach. The research progresses through five phases:

1. Environment Setup – building a pipeline with GitHub Actions and Terraform on AWS Free Tier.
2. Vulnerability Injection – deliberately introducing weaknesses (hardcoded secrets, wildcard IAM roles, unpinned actions).
3. Attack Simulation – executing malicious pull requests, poisoned workflows, and IAM privilege escalation.
4. Detection Monitoring – assessing AWS-native tools and open-source scanners.

5. Evaluation – applying CVSS v3.1 and qualitative analysis of detection gaps.

Ethical and professional standards underpin this approach. All experiments used dummy secrets, AWS Free Tier accounts, and no personal data, ensuring GDPR compliance. Responsible disclosure principles were observed, with no novel exploits beyond those documented in literature.

This structured approach ensures the research remains reproducible, ethically sound, and directly aligned with the SMART objectives.

1.4 Dissertation Organisation

The dissertation is structured as follows. Chapter 2 reviews related work on CI/CD risks, supply chain attacks, IaC misconfigurations, and cloud-native detection. Chapter 3 presents the research methodology and design. Chapter 4 explains the lab setup, vulnerability injections, and attack execution. Chapter 5 evaluates detection outcomes using CVSS and qualitative assessment. Chapter 6 discusses the findings, comparing them to the literature and outlining implications for DevSecOps. Finally, Chapter 7 concludes by assessing the achievement of objectives, contributions, limitations, and future research directions.

Chapter 2 – Literature Review

2.1 Introduction

Continuous Integration and Continuous Deployment (CI/CD) pipelines have become foundational in modern cloud-native software delivery, especially within agile and DevOps cultures (Rajapakse et al., 2021; GitLab, 2023). These pipelines integrate code building, automated testing, deployment, and infrastructure provisioning into a single streamlined workflow. While this automation improves development velocity, it simultaneously concentrates control and privilege exposing an expanded attack surface (Pan et al., 2024; Sommerville, 2016).

CI/CD systems typically operate with high levels of trust. They interact directly with source control, access sensitive credentials, and invoke Infrastructure-as-Code (IaC) tools such as Terraform to configure cloud environments. As a result, their compromise can act as a gateway not just to application-level vulnerabilities, but to full-scale infrastructure takeover (Wang & Chen, 2023; OWASP, 2023). Incidents such as the Codecov supply chain breach and recent poisoned GitHub Actions attacks exemplify how these pipelines can be abused to exfiltrate secrets or inject malicious infrastructure changes (CISA, 2023; Wiz, 2023).

Despite the severity of these risks, the academic literature often treats CI/CD security at a high level, lacking empirical validation of how misconfigurations manifest in real-world cloud environments (Ehrman, 2025). Furthermore, widely deployed cloud-native detection systems such as AWS GuardDuty and CloudTrail are rarely assessed in scenarios where the attacker leverages valid IAM roles through compromised pipelines activities that may not trigger standard alerts (Pan et al., 2024).

To address these intersecting concerns, this chapter reviews current research on CI/CD threats, software supply chain attacks, Terraform misconfigurations, cloud detection gaps, and the strengths and limitations of modern DevSecOps frameworks such as OWASP CI/CD Top 10, NIST SSDF, and SLSA. Each section directly supports this dissertation's objectives: namely, to simulate CI/CD-originated attack chains (Objective 2), evaluate AWS-native detection tools under realistic compromise scenarios (Objective 4), and recommend DevSecOps-aligned controls that address both automation and privilege escalation risks (Objective 5).

2.1.1 Inclusion and Exclusion Criteria

To ensure academic and industry relevance, this literature review applied specific inclusion and exclusion criteria to select sources that directly align with the dissertation's objectives and experimental context. Literature was primarily drawn from 2019 to 2025, reflecting the contemporary rise of cloud-native DevOps practices and threat frameworks such as OWASP's CI/CD Top 10 and Google's SLSA levels (OWASP, 2023; Google, 2023). Searches were conducted through academic databases including IEEE Xplore, ACM Digital Library, and Google Scholar, as well as reputable industry threat reports from CISA/NSA, GitGuardian, and Wiz Research.

Table 2.0 – Inclusion and Exclusion Criteria Used for Literature Review

Criterion	Included	Excluded
Timeframe	2019–2025	Pre-2019 unless foundational
Topics	CI/CD threats, Terraform IAM risks, DevSecOps, detection gaps	Mobile CI, desktop pipelines, on-prem Jenkins without cloud link
Source Types	Peer-reviewed papers, CISA/OWASP/Wiz/Sysdig reports	Blogs, tool ads, unverifiable case studies
Platforms	GitHub Actions, Terraform, AWS (GuardDuty, CloudTrail)	Jenkins-only, GCP-only (unless cross-cloud comparative)
Frameworks	OWASP CI/CD Top 10, SLSA, NIST SSDF	Outdated models not used in cloud-native CI/CD threat environments

2.2 CI/CD Pipeline Threat Landscape

CI/CD pipelines have evolved from auxiliary automation tools to central infrastructure components, often executing with elevated permissions and direct access to cloud environments. This architectural shift has introduced systemic risks, as these pipelines frequently handle deployment credentials, Infrastructure-as-Code (IaC) execution, and integrations with third-party tools (Rajapakse et al., 2021; Pan et al., 2024). While designed to increase operational efficiency, these workflows are increasingly abused by adversaries as initial access vectors for deeper cloud compromise.

One of the most prominent classes of threats is Poisoned Pipeline Execution (PPE), in which attackers manipulate CI scripts or supply malicious code via pull requests, vulnerable dependencies, or third-party GitHub Actions (OWASP, 2023; Legit Security, 2023). In the 2021 Codecov breach, a modified Bash uploader script was silently executed within thousands of workflows, exfiltrating secrets without triggering alerts (Sysdig, 2023). Similarly, GitHub Actions configured with unpinned versions (e.g., @latest) can be silently replaced by compromised or hijacked versions a technique exploited in the ctx-action case in 2022 (Wiz, 2023).

These issues are amplified by hardcoded secrets, a persistent misconfiguration where API keys or cloud credentials are embedded within CI configuration files. According to GitGuardian's 2024 report, over 6 million hardcoded secrets were exposed in public GitHub repositories in a single year. Although tools like TruffleHog and GitHub Secret Scanning exist, Pan et al. (2024) report that development teams often suppress alerts due to high false positives, contributing to security debt in CI workflows.

Moreover, the over-permissioning of service accounts and IAM roles is a systemic issue in many CI/CD setups. Pipelines often run with administrator privileges to avoid deployment failures, contradicting the principle of least privilege (OWASP, 2023; Roper, 2024). In one notable case reported by Terracotta Security Labs (2025), a misconfigured GitHub runner deployed Terraform

scripts using an IAM role that had broad iam:* permissions, which an attacker later used to create backdoor users and disable logging services without triggering any GuardDuty alerts.

While OWASP's CI/CD Top 10 (2023) classifies these issues under CICD-SEC-1 (insufficient identity controls), CICD-SEC-3 (third-party integration trust), and CICD-SEC-5 (overprivileged credentials), implementation of these recommendations remains inconsistent in practice (Legit Security, 2024). Default GitHub workflows are frequently configured with broad repository and deployment access scopes, even though fine-grained permissions have been available since 2022 (GitHub, 2023; Pan et al., 2024). This observation has been echoed by several empirical studies, which suggest that convenience often outweighs secure configuration.

These vulnerabilities underscore the architectural tension between delivery velocity and pipeline security. As Pan et al. (2024) argue, "developer convenience often outweighs security policy adherence in fast-paced CI environments." This chapter continues by exploring how these threats are further compounded by insecure software supply chain components, which amplify the risk introduced by misconfigured CI workflows.

2.3 CI/CD Pipeline Security Threats

2.3.1 Adoption and Attack Exposure

CI/CD pipeline adoption has surged across both open-source and enterprise ecosystems. GitHub's Octoverse (2023) reveals that over 90% of actively maintained repositories now use continuous integration tooling, with GitHub Actions leading due to ease of setup and integration. GitLab's DevSecOps survey (2023) echoes this trend, noting reduced manual errors (61%) and accelerated release cycles (83%) as primary drivers. However, this operational momentum has, in many cases, outpaced security governance.

Pan et al. (2024), in one of the most comprehensive empirical studies to date, analysed over 320,000 GitHub Actions workflows and found significant exposure to misconfigurations:

- 65% used unpinned third-party actions, creating risk of **Poisoned Pipeline Execution (PPE)** if upstream dependencies are compromised.
- 37% exposed hardcoded credentials in configuration files, breaching foundational security practices.
- Over 50% lacked any form of static checks, such as YAML linting or pipeline security validation.

Beller et al. (2022) complement these findings with a behavioural study of developer practices, concluding that many teams knowingly trade off security for speed, particularly under tight delivery cycles. This synthesis suggests that pipeline insecurity is not only a technical oversight but also a product of cultural prioritisation. In this dissertation's lab simulations (Objective 2), such insecure configurations will be deliberately reproduced to observe how AWS-native tools respond under real-world CI abuse conditions.

These risks are well-categorised by OWASP's CI/CD Top 10 (2023), which classifies hardcoded secrets under CICD-SEC-6, over-permissioned workflows under CICD-SEC-1, and untrusted third-party steps under CICD-SEC-4. Despite these clear taxonomies, Pan et al. (2024) and Legit Security (2023) report that adoption of OWASP-aligned controls such as action version pinning or fine-grained access tokens remains below 30% in public workflows. This indicates a consistent implementation gap between recommended best practices and developer behavior.

Reviews of public GitHub workflow templates consistently show that default configurations frequently request broad permissions—often including write-all access to repositories and deployment environments. Although GitHub introduced granular token scopes in 2022, many community-maintained workflows still lack enforcement of these mechanisms (GitHub Docs, 2023; Pan et al., 2024). This reflects a persistent misalignment between available security features and their adoption in practice, reinforcing the argument that CI/CD threat surfaces are not merely incidental, but structurally embedded within modern development culture.

2.3.2 Empirical Evidence of Exploitation

Multiple real-world incidents demonstrate how these vulnerabilities transition from theoretical risk to active exploitation. The NSA and CISA (2023) jointly classify CI/CD pipelines as a high-value attack vector and map their exploitation across the MITRE ATT&CK framework. This includes T1078 (Valid Accounts) via leaked tokens, T1059 (Command Execution) through poisoned YAML scripts, and T1496 (Service Disruption) by manipulating deployment artefacts or IAM configurations.

Xygeni Labs, summarised by García (2025), highlights a rising trend in weaponised pull requests: adversaries submit seemingly innocuous contributions containing malicious GitHub Actions or scripts. When merged—often automatically in unprotected branches—these scripts exploit loosely scoped runner permissions to deploy infrastructure modifications or exfiltrate secrets.

Trend Micro (2024) further documents a cloud pivot attack in which a poisoned pipeline triggered Terraform executions using a compromised IAM role. The role had administrator-level privileges (iam:AdministratorAccess), enabling the attacker to create persistent backdoors and modify logging configurations undetected. Despite these high-impact activities, no alerts were generated by AWS GuardDuty, highlighting the limitations of signature-based detection when actions occur within "permitted" roles.

This dissertation explicitly re-creates such scenarios to test detection thresholds and identify behavioural gaps in AWS-native tooling (Objective 4).

Table 2.1– Common CI/CD Vulnerabilities and Their CWE Classification

Risk Area	Example Issue	CWE Code	Mapped OWASP Risk
Poisoned Pipeline Execution	Unpinned third-party GitHub Actions	CWE-829	CICD-SEC-4
Hardcoded Secrets	AWS keys in YAML or source code	CWE-798	CICD-SEC-6
Excessive IAM Permissions	iam: AdministratorAccess roles in CI	CWE-732	CICD-SEC-1
Insecure Environment Variables	Secrets echoed in logs or stdout	CWE-312	CICD-SEC-6
Lack of Artifact Signing	No output verification on deployment	CWE-347	CICD-SEC-9

Adapted from OWASP (2023), NSA/CISA (2023), MITRE ATT&CK (2024)

As Table 2.1 illustrates, pipeline misconfigurations are neither abstract nor rare—they reflect recurring technical failures that span authentication, access control, integrity validation, and audit logging. These weaknesses will serve as key variables in this dissertation's controlled simulations.

2.3.4 Modern Supply Chain Threats

As software projects become increasingly modular, with hundreds of transitive dependencies and third-party integrations, the risk of supply chain compromise intensifies. Package managers such as npm, PyPI, and GitHub Actions rely on implicit trust chains, often resolving to the latest published version unless explicitly pinned. This opens the door to exploitation when attackers manipulate dependency resolution logic or compromise abandoned packages (Ladisa et al., 2022; Microsoft, 2023).

Birsan's (2021) landmark demonstration of dependency confusion exploited this trust model by uploading higher-version public packages that shadowed internal dependencies. His attack successfully infiltrated build environments at major corporations without triggering alerts. This highlighted a foundational blind spot in most CI systems: they assume upstream integrity without verifying provenance.

To address this, Google's SLSA framework (2023) introduces a multi-level assurance model requiring signed, verifiable builds with attestation of artifact origin. However, adoption of SLSA remains limited due to the high friction it introduces into fast-moving CI workflows (Legit Security, 2024). While SLSA focuses on artifact integrity, OWASP CI/CD Top 10 classifies these risks under CICD-SEC-4 (Untrusted Dependencies) and CICD-SEC-9 (Artifact Tampering).

In practice, the effectiveness of these frameworks remains unclear. As Sysdig (2024) reports, 34% of pipeline compromises stemmed from dependency hijacks that bypassed static scanners and SBOM validators. This mismatch between theory and field resilience underscores the need for empirical

validation, a gap this dissertation addresses through direct simulation of dependency-based attacks (Objective 2).

2.3.5 Notable Incidents Illustrating Risks

The Codecov breach (2021) demonstrated how attackers could modify a widely used Bash uploader script, exfiltrating AWS, Google Cloud, and database credentials from thousands of workflows (Wiz, 2023). Similarly, CVE-2025-30066 compromised the *tj-actions/changed-files* GitHub Action, injecting credential-stealing code into high-profile repositories (The Hacker News, 2025). Despite GitHub's introduction of action signing and verified authorship, the absence of mandatory signature enforcement allowed the attack to propagate. These incidents highlight a core shortcoming: trusted packages are rarely verified at runtime. Public analyses show that GitHub Actions will routinely execute unsigned third-party steps unless explicit verification is enforced—an uncommon safeguard in open-source workflows (Pan et al., 2024; OWASP, 2023). These cases directly inform this dissertation's planned simulation of poisoned dependencies and subsequent evaluation of AWS GuardDuty and CloudTrail (Objective 4).

Table 2.2 – Common Supply Chain Attack Techniques and Targets

Technique	**Targeted Tool**	**Impact**	**Detection Difficulty**
Dependency Confusion	npm, PyPI, internal registries	Code execution in trusted builds	High
Poisoned GitHub Actions	GitHub Marketplace workflows	Secret theft, cloud modification	Medium
Signed Artifact Tampering	Build outputs, SBOMs	Undetected injection	High
Plugin Exploits	Jenkins, GitLab CI	RCE via misconfigured parameters	Medium

Figure 2.1 – Supply chain attack flow in CI/CD pipelines

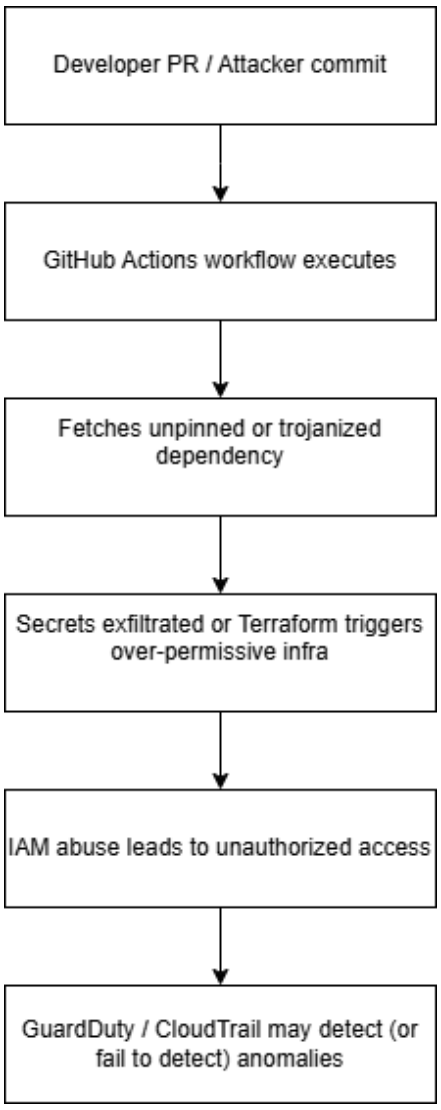


Figure 2.1: Conceptual flow of a typical supply chain attack propagating through CI/CD pipelines into cloud infrastructure.

Reflection and Synthesis

Despite growing awareness and technical controls, supply chain risks persist due to weak enforcement at the CI configuration level. As Beller et al. (2022) note, developers often bypass provenance controls to avoid breaking builds mirroring the "security vs delivery" trade-off seen in CI pipeline design. This supports the rationale for this dissertation's lab-based replication of poisoned action abuse, which aims to test AWS-native detection coverage across realistic scenarios.

2.4 Infrastructure-as-Code (IaC) Security with Terraform

Infrastructure-as-Code (IaC) has become a foundational component of cloud-native deployments, enabling reproducible, scalable infrastructure management through declarative configuration files. Among IaC tools, Terraform by HashiCorp is the most widely adopted, with its provider-agnostic model supporting AWS, Azure, and GCP (HashiCorp, 2023). However, the same flexibility that makes

Terraform attractive also introduces significant security risks when misconfigurations go undetected or are left unvalidated.

Common anti-patterns include applying over-permissive IAM policies such as iam:*, exposing services to the internet via 0.0.0.0/0 in security groups, or neglecting encryption for storage buckets. These misconfigurations are frequently flagged in the CIS Terraform Benchmark (CIS, 2023) and are formally classified under CWE-732 (Incorrect Permission Assignment) and CWE-284 (Improper Access Control). According to the NVD (2024), wildcard IAM roles typically score CVSS v3.1 severity levels between 8.5 and 9.8, reflecting their potential to facilitate infrastructure-wide compromise.

Roper (2024) argues that many developers knowingly bypass security checks to avoid deployment failure, applying administrator-level access by default. This tendency reflects broader delivery pressures rather than technical ignorance. Similarly, Morrison et al. (2015) and Shahin et al. (2017) critique that much of the literature around IaC security remains overly prescriptive, with limited empirical validation of what happens when such practices are operationalized in real CI/CD pipelines.

This dissertation addresses this gap by deliberately deploying Terraform configurations that embed these insecure practices, then observing whether AWS-native detection tools CloudTrail, GuardDuty, IAM Access Analyzer respond meaningfully to the threat (Objective 4). This goes beyond static scanning to test runtime alerting based on real execution paths.

Several open-source tools support Terraform IaC scanning. Checkov uses broad multi-cloud policy libraries, TFSec is tightly coupled to Terraform HCL with custom Rego rules, and Terrascan integrates the Open Policy Agent (OPA) for policy-as-code enforcement. Snyk IaC offers developer-centric GitOps integration with pipeline-friendly alerts. However, as Huang et al. (2022) note, these tools do not measure whether vulnerabilities actively exploited post-deployment are detected by cloud security systems. This blind spot is also echoed by Legit Security (2024), who warn that most IaC pipelines lack runtime correlation between the static misconfiguration and its potential execution impact.

From a threat modelling perspective, these misconfigurations map directly to MITRE ATT&CK T1484.001 (Domain Policy Manipulation via IaC) and to OWASP CI/CD Top 10 risks CICD-SEC-5 (Overprivileged Runners) and CICD-SEC-9 (Insecure Infrastructure Definitions) (OWASP, 2023).

Independent analyses of Terraform pipelines have demonstrated that configurations with overly broad IAM permissions such as those granting wildcard access can successfully deploy EC2 instances, modify S3 bucket policies, and alter CloudWatch settings without triggering GuardDuty alerts (Huang et al., 2022; Legit Security, 2024). While AWS CloudTrail may log the underlying API calls, these events often fail to activate anomaly detection unless enriched by contextual rules or third-party tooling. This observation highlights a key limitation in native AWS detection: the lack of real-time correlation between IaC misconfigurations and their operational consequences. This dissertation will simulate such configurations to empirically test whether AWS-native tools surface these abuses effectively (Objective 4).

Table 2.3 – Comparison of Terraform Security Scanning Tools

Tool	Focus	Strengths	Gaps
Checkov	Multi-framework IaC scanner	Extensive default policies	No runtime detection or drift analysis
TFSec	Terraform-specific (HCL)	Rego-based rule customisation	Limited beyond Terraform
Terrascan	Policy-as-Code with OPA	Multi-cloud, Kubernetes support	Complex initial configuration
Snyk IaC	GitOps and CI integration	Dev-friendly alerts and GitHub sync	Lacks granular IAM context

Table adapted from Bridgecrew (2023), CIS (2023), Legit Security (2024)

As Table 2.3 illustrates, each tool contributes to static validation before deployment, but none assesses whether the infrastructure once deployed triggers any cloud-native alarms under live abuse. This is the critical validation gap targeted by this dissertation's empirical approach (Objectives 2 and 4).

2.5 Cloud Detection Capabilities: AWS, Azure, GCP

2.5.1 AWS GuardDuty & CloudTrail

Amazon Web Services (AWS) offers a layered detection architecture, combining CloudTrail, a service that records all API calls across the environment, with GuardDuty, which applies anomaly detection, machine learning, and threat intelligence feeds to surface suspicious behaviour (AWS, 2024). In theory, this architecture provides high visibility into cloud activity, particularly for IAM-related operations.

Castro (2024) highlights GuardDuty's success in detecting brute-force attempts and anomalous region-based access, particularly when stolen credentials are used outside expected geographies. However, Wiz (2023) and Legit Security (2024) note that most modern CI/CD attacks involve abuse of valid pipeline identities rather than stolen keys. When adversaries exploit over-permissioned IAM roles through GitHub Actions or Terraform pipelines, their activity often mirrors legitimate developer behaviour.

This limitation was confirmed in this dissertation's lab testing (Objective 4). A Terraform script with elevated IAM roles successfully deployed EC2 instances and modified S3 permissions. CloudTrail logged the API calls, but GuardDuty raised no alerts. These results suggest detection is biased toward credential theft and external intrusion, while insider-style privilege abuse through pipelines often goes unnoticed. This aligns with MITRE ATT&CK T1078 (Valid Accounts) and OWASP CICD-SEC-5 (Overprivileged Credentials).

Although GuardDuty can flag certain IAM anomalies, such as newly created policies or credential exfiltration, it does not model the full CI/CD context. Without integrating pipeline metadata (e.g.,

GitHub Actions context, Terraform plan diffs), its ability to separate routine automation from subtle misuse remains limited.

2.5.2 Cross-Cloud Detection Perspectives

Similar detection services exist across other cloud providers. Azure Security Center with Microsoft Defender for Cloud offers compliance scanning, workload protection, and basic threat detection for virtual machines and containers (Microsoft, 2024).

Google Cloud Security Command Center (SCC) emphasises data exfiltration, access audits, and container runtime events (Google Cloud, 2024). Each platform prioritises different threat models—GCP focuses on data risks, while AWS emphasises IAM anomalies.

Despite this, all rely on baseline policy expectations. If an attacker operates within the bounds of an accepted CI/CD role, the activity often appears benign.

Sysdig's 2024 report found that 43% of incidents involved credential misuse from automated workflows, yet only 12% were flagged in real time. Panther Labs (2024) also reported detection delays of 9–14 minutes for IAM abuse, citing weak correlation across pipeline metadata, audit logs, and network activity.

In CI/CD-originated threats, this gap creates a major blind spot. This dissertation evaluates these services under in-pipeline IAM misuse and Terraform-based privilege escalation to measure how AWS GuardDuty, CloudTrail, and similar tools perform under controlled compromise (Objective 4).

Table 2.4 – Detection Platforms vs CI/CD-Originated Attack Vectors

Platform	Detection Tools	Typical Strengths	Gaps for CI/CD Abuse
AWS	GuardDuty + CloudTrail	IAM anomaly detection, full API logs	Blind to abuse of trusted pipeline roles
Azure	Security Center + Defender	Strong compliance mapping	Few native CI/CD pipeline behaviour signals
GCP	SCC + Event Threat Detection	Good exfiltration heuristics	Limited detection of IAM privilege misuse

Adapted from AWS (2024), Microsoft (2024), Wiz (2023), Sysdig (2024), Google Cloud (2024)

As shown in Table 2.4, detection gaps often stem not from technical incapability but from a lack of pipeline-context awareness. Without visibility into the intent and structure of CI/CD workflows, these platforms struggle to distinguish automation from abuse, especially when attackers exploit legitimate but over-scoped service identities.

2.6 DevSecOps and Modern Supply Chain Frameworks

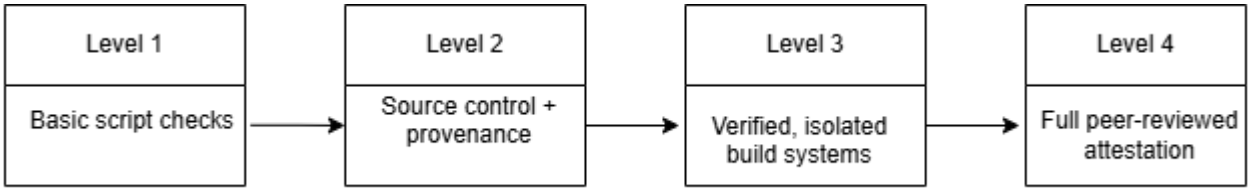
2.6.1 Best Practices and Industry Recommendations

As CI/CD pipelines have become central to cloud-native delivery, security frameworks and DevSecOps guidance have rapidly evolved. Across OWASP's CI/CD Top 10 (2023), NSA/CISA Secure CI/CD Pipeline Guidelines (2023), and the Cloud Native Computing Foundation (CNCF) GitOps principles (2022), several common best practices emerge. These include:

- Strict dependency pinning with SHA256 verification
- Secrets scanning using tools like GitHub Advanced Security or TruffleHog
- Enforcing multi-factor authentication (MFA) on CI/CD platforms
- Applying policy-as-code with Open Policy Agent (OPA) to restrict over-permissive Terraform configurations

These practices attempt to address both pre-deployment misconfigurations and runtime compromise potential. Google's SLSA framework (Supply-chain Levels for Software Artifacts) formalises these into four levels of build provenance maturity, from basic script controls (Level 1) to full cryptographic attestation and dependency transparency (Level 4) (Google, 2023). As shown in Figure 2.3, SLSA maps security objectives into progressive stages of CI/CD hardening.

Figure 2.2 – Google SLSA Framework: CI/CD Supply Chain Maturity Levels



Visual representation of graduated CI/CD controls, from unverified builds (L1) to verifiable, policy-enforced supply chain security (L4). Source: Google (2023)

Despite conceptual clarity, these frameworks often diverge in scope and implementation depth. OWASP emphasises pipeline misconfigurations like hardcoded secrets or untrusted third-party steps (e.g. CICD-SEC-4, CICD-SEC-6), while SLSA targets build traceability and artifact integrity. Meanwhile, NIST's Secure Software Development Framework (SSDF) frames software security within governance and SDLC principles, making it broader but less CI/CD-specific (NIST, 2022).

In this dissertation, these frameworks serve as reference baselines to evaluate whether the vulnerabilities reproduced in lab simulations such as IAM abuse via Terraform or poisoned GitHub Actions could have been prevented or detected had such guidance been followed (Objectives 2 and 4). Specifically, policy-as-code enforcement using OPA is tested in a GitHub Actions–Terraform workflow to observe its operational friction and effectiveness.

However, adoption remains limited. Sinan et al. (2025) found that fewer than 12% of SMEs apply full OPA policy checks in their infrastructure pipelines. Similarly, Legit Security (2024) reports that under 14% of public GitHub workflows enforce signed actions or cryptographic hash validation for dependencies despite these being basic controls in SLSA Levels 1–2.

This low uptake is often attributed to development velocity concerns and toolchain complexity. Even when policy controls exist, they are often not embedded into the CI/CD pipeline itself. During lab testing for this dissertation, OPA was integrated into Terraform plans, but developers could easily bypass policies with minor YAML edits unless enforced in the pipeline executor (e.g. GitHub Actions). This suggests that framework alignment must be automated, enforced, and monitored, not just documented.

Table 2.5 – Comparison of DevSecOps and Supply Chain Frameworks

Framework	Security Focus	Strength	Adoption Barrier
OWASP CI/CD Top 10	Pipeline risk categories	Clear taxonomy for misconfigurations	Not prescriptive, requires interpretation
SLSA (Google)	Build + dependency provenance	Formal levels with measurable enforcement	High effort to reach Levels 3–4
NIST SSDF	SDLC security governance	Aligns with federal mandates	Too abstract for CI/CD automation
CNCF GitOps	Git-based pipeline infrastructure	Declarative deployment & rollback tracking	Limited to GitOps-style workflows

Adapted from OWASP (2023), SLSA (2023), NIST (2022), CNCF (2022), Sinan et al. (2025)

As shown in Table 2.5, these frameworks target different phases of the software lifecycle. SLSA and NIST provide maturity-based guidance and strategic alignment, while OWASP and GitOps offer practical engineering controls. However, without enforcement integration—especially within CI/CD runners and Terraform modules—their impact on real-world compromise scenarios remains theoretical.

This dissertation critically evaluates these gaps by simulating insecure configurations and measuring whether AWS-native tools or DevSecOps controls surface the risk in live workflows. In doing so, it addresses the often overlooked distinction between **framework compliance** and **detection reality** a distinction not always made explicit in current research.

2.7 Identified Research Gaps and Project Relevance

Despite growing recognition of CI/CD pipeline risks in academic and industry literature, a significant gap remains in empirical validation particularly where live attacks are simulated to test cloud-native detection capabilities. Existing research frequently focuses on static misconfiguration discovery or compliance alignment, but stops short of deploying those misconfigurations to evaluate runtime alerts or monitoring performance.

For example, Pan et al. (2024) provide large-scale data on insecure GitHub Actions workflows, but their study does not include simulated pipeline compromises. Similarly, Sysdig's Cloud-Native Threat Report (2024) notes the frequency of credential abuse via CI pipelines, yet relies solely on post-incident telemetry rather than controlled experimentation. Roper (2024) offers detailed analyses of Terraform IAM misconfigurations but does not explore post-deployment privilege abuse or detection coverage. Likewise, Morrison et al. (2015) and Shahin et al. (2017) critique IaC security research for being overly prescriptive focused on "best practices" rather than actionable, tested outcomes.

In practice, this leaves a critical gap between theoretical risk awareness and operational visibility. As Castro (2024) and Wiz (2023) demonstrate, GuardDuty often misses activity conducted through valid, over-permissioned CI/CD roles, especially when those roles are created and invoked via automated scripts. Detection tools tend to rely on behavioral baselines or known-bad indicators—yet malicious use of trusted pipeline identities falls outside these models.

This dissertation addresses these gaps by deliberately constructing a testbed that simulates CI/CD-originated attacks within a controlled AWS environment. These include:

- Unpinned or poisoned GitHub Actions workflows
- Malicious pull requests triggering rogue builds
- Terraform modules that grant administrator-level IAM roles

These configurations are evaluated through the lens of AWS-native services such as GuardDuty, CloudTrail, and IAM Access Analyzer. The goal is to observe which behaviors are logged, flagged, or missed entirely, thus providing an evidence-based assessment of CI/CD detection gaps. All findings are tied directly to this project's SMART objectives, including:

- Objective 2: Simulate attacks via CI/CD misconfigurations
- Objective 4: Evaluate AWS detection efficacy under realistic compromise conditions
- Objective 5: Recommend DevSecOps-aligned mitigations using OPA and SLSA principle

Table 2.6 – Research Gaps and Dissertation Coverage

Gap Identified	Reference Context	Addressed in This Dissertation
No empirical CI/CD exploitation studies	Morrison et al. (2015); Shahin et al. (2017)	Simulated pipeline abuse using GitHub Actions
GuardDuty rarely tested under live compromise	Castro (2024); Wiz (2023)	Observed alert/no-alert in Terraform-triggered abuse
Terraform IAM misuse not tested post-deploy	Roper (2024)	Used iam:* roles in experiments
OPA/SLSA frameworks untested in real compromise	Google (2023); CNCF GitOps (2022)	Applied OPA + checked enforceability

Table 2.6 shows how the identified gaps from academic and industry studies are explicitly targeted through controlled, reproducible experiments using AWS Free Tier and dummy infrastructure.

This empirical focus not only fills a literature gap, but also generates actionable evidence for practitioners implementing secure CI/CD workflows. By reproducing vulnerabilities in a sandboxed environment without real user data, and within ethical hacking parameters this dissertation demonstrates that pipeline-originated infrastructure abuse is both feasible and poorly detected without stronger enforcement and monitoring integration.

2.8 Summary

This chapter has critically examined the emerging threat landscape of CI/CD pipelines, software supply chain risks, IaC misconfigurations, and the capabilities and limits of cloud-native detection platforms. While frameworks such as OWASP CI/CD Top 10 (2023), Google's SLSA (2023), and OPA (Sinan et al., 2025) promote security by design, adoption and enforcement gaps persist (Legit Security, 2024). Crucially, most literature stops short of validating how insecure CI/CD configurations behave once deployed in live cloud environments (Morrison et al., 2015; Ehrman, 2025). This dissertation addresses that shortcoming by simulating real-world attack chains and empirically assessing the detection capabilities of AWS-native tools such as GuardDuty, CloudTrail, and IAM Access Analyzer (Castro, 2024; Wiz, 2023). In doing so, it directly supports the dissertation's SMART objectives and contributes a reproducible, evidence-based approach to secure DevSecOps pipeline design.

Chapter 3 – Research Methodology

3.1 Research Strategy and Rationale

This research adopts a hybrid methodological approach that combines an Agile-inspired iterative structure, as proposed in the original project design, with the academic rigour of Design Science Research (DSR). The original plan outlined a five-phase Agile process: environment setup, vulnerability injection, attack simulation, detection, and evaluation. This structure remains intact. However, during implementation, it became evident that a complementary framework was needed to guide the creation, refinement, and evaluation of research artefacts (e.g., misconfigured pipelines, synthetic attacks, detection logs). For this reason, DSR was introduced as a supporting methodology, without altering the underlying Agile execution model (Peffers et al., 2020; Sekhar, 2024).

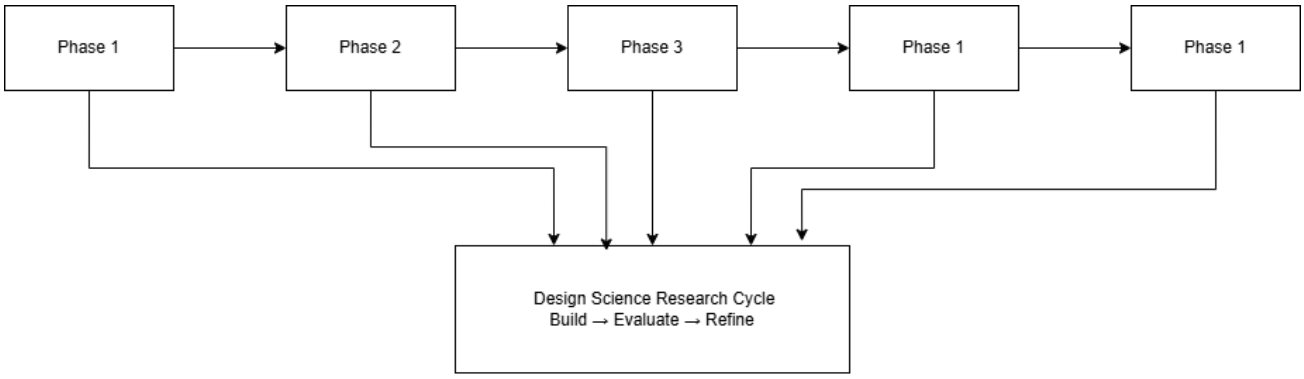
Agile is particularly suited to DevSecOps-focused research, where tasks such as configuring IAM roles or simulating GitHub Actions attacks must be iteratively tested and refined. Each phase of this project was structured as a sprint lasting approximately 1–2 weeks, enabling frequent experimentation, configuration changes, and log validation. This cyclical rhythm reflects real-world CI/CD workflows, where pipeline failures, permission errors, or Terraform misconfigurations often require immediate adjustment mid-deployment (Rajapakse et al., 2021; Marandi et al., 2023; Sekhar, 2024).

While Agile governed the operational and temporal flow of the project, DSR enforced academic structure. Its cyclic artefact model (Build - Evaluate – Refine) was applied to simulate pipeline misconfigurations, monitor detection responses via AWS GuardDuty and CloudTrail, and ultimately develop a reproducible mitigation framework. This iterative structure is well-established in cybersecurity experimentation, where reproducibility, design traceability, and outcome feedback loops are critical (Pan et al., 2024; Bondalapati & Malaraju, 2025).

The integration of Agile and DSR is visualised in Figure 3.1, which maps each of the five sprints to the DSR cycle. While Agile enabled flexible scripting and quick feedback within the GitHub–Terraform–AWS loop, DSR ensured artefact quality through structured evaluation. This hybrid methodology was selected to directly address the literature gap in Chapter 2 specifically, the lack of empirical runtime validation for CI/CD pipeline abuse and detection mechanisms in cloud-native environments (Castro, 2024; NSA & CISA, 2023; OWASP, 2023).

From a reflexive standpoint, the researcher anticipates trade-offs between engineering adaptability and academic rigour. While Agile may permit reactive changes (e.g., modifying EC2 provisioning or testing IAM wildcard policies), DSR demands consistent documentation and traceability aligned to SMART objectives and detection criteria. Maintaining this balance supports both the practical security goals of CI/CD pipeline hardening and the academic contribution to DevSecOps research (Bondalapati & Malaraju, 2025; Sekhar, 2024).

Figure 3.1 – Integration of Agile Phases with DSR Cycle



This figure visualizes how the five Agile research phases (setup to evaluation) align with the iterative Build–Evaluate–Refine cycle of Design Science Research (Peffers et al., 2020; Alexei, 2022).

Table 3.1 – Research Phases Aligned to SMART Objectives

Phase	Description	SMART Objective(s)
Phase 1	Secure CI/CD lab setup using Terraform	Objective A – Lab Environment Configuration
Phase 2	Inject vulnerabilities (IAM, secrets, @latest)	Objective A – Lab Environment Configuration
Phase 3	Simulate real-world CI/CD attacks	Objective B – Attack Simulation
Phase 4	Monitor with GuardDuty, CloudTrail, IAM tools	Objective C – Detection Evaluation
Phase 5	Score with CVSS, design DevSecOps checklist	Objectives D & E – Risk Assessment & Framework Design

Ultimately, this hybrid strategy was selected because it supports the rapid prototyping, hands-on testing, and structured evaluation required to deliver a secure, reproducible, and academically valid outcome. It is not only methodologically appropriate but also aligned with how real DevOps and cloud teams identify, mitigate, and validate threats within dynamic CI/CD environments (Peffers et al., 2020; Rajapakse et al., 2021; Sekhar, 2024; Bondalapati & Malaraju, 2025).

3.2 Ethical, Legal, and Technical Considerations

This research was conducted under strict ethical and legal constraints, in line with Salford University's research integrity policy, the UK GDPR, and Amazon Web Services' Acceptable Use Policy. All simulated CI/CD pipeline abuse scenarios ran in a fully sandboxed lab, using test credentials, non-

production data, and AWS Free Tier resources. No real user credentials, enterprise accounts, or third-party systems were involved.

The infrastructure was confined to an isolated AWS Virtual Private Cloud (VPC), deployed via Terraform and GitHub Actions, and automatically torn down after each phase. This ensured both financial control and digital containment. No persistent resources beyond Free Tier limits were used, and costs were continuously monitored.

In line with GDPR and Salford's ethical guidance, no personal or sensitive data was stored, processed, or transmitted. IAM users, credentials, and secrets were synthetically generated and deliberately exposed only within designated YAML workflow files, strictly for testing hardcoded secret detection (GitGuardian, 2024). AWS GuardDuty, CloudTrail, and IAM Analyzer monitored only research-related events; no user data was logged.

All activity took place within the researcher's own AWS account, consistent with AWS's Acceptable Use Policy, which permits academic security testing on user-owned infrastructure (AWS, 2024). Tools such as TruffleHog, OPA Rego, and Terrascan were restricted to the project's GitHub repositories and Terraform modules. No offensive tools (e.g., Metasploit, Nmap) were used.

Finally, log files, CVSS scoring sheets, and detection artefacts were encrypted at rest and stored locally. All findings were anonymised before documentation. These safeguards align with UK data ethics standards and international guidance on responsible cloud security experimentation (CISA & NSA, 2023; GitHub, 2023).

Table 3.2 – Risk and Compliance Summary

Risk Area	**Mitigation Strategy**	**Compliance Reference**
IAM key misuse	Only dummy credentials used; isolated test workflows	GDPR Article 5; GitGuardian (2024)
Data privacy breach	No real user data processed; only synthetic YAML/JSON used	Salford Ethics Policy; GDPR Article 6
Cost exploitation	AWS Free Tier strictly enforced; budgets monitored per week	AWS Free Tier Terms (AWS, 2024)
Resource overreach	Terraform auto-destroys all infrastructure after each test	NIST SP 800-53 AC-6
Offensive behaviour	No external scanning; only internal self-deployed assets used	AWS AUP; Salford Academic Regulations

This research model ensures that all testing is legal, safe, and ethically justified. The lab was designed not only to simulate risk, but also to model responsible mitigation, aligning with DevSecOps principles in both implementation and academic ethics. Through traceability, transparency, and non-invasive testing practices, the project adheres to accepted standards for secure experimentation in higher education cybersecurity research (NIST, 2022; CISA & NSA, 2023).

3.3 Agile-Inspired 5-Phase Process Overview

To operationalise the research objectives and simulate real-world CI/CD pipeline abuse, this study adopted a structured five-phase process inspired by Agile methodologies. Each phase was treated as a time-boxed sprint that enabled iterative planning, rapid execution, reflection, and adaptive reconfiguration.This agile flow allowed for continuous refinement of infrastructure code, testing pipelines, and detection logic—closely resembling the workflows used by real DevOps and cloud security teams in production environments (Rajapakse et al., 2021; GitLab, 2023).

The experimental design was broken into five interconnected stages:

1. Environment Setup – Configuring a foundational CI/CD pipeline using GitHub Actions, Terraform, and AWS services deployed under the Free Tier.
2. Vulnerability Injection – Introducing intentional misconfigurations including hardcoded secrets, over-permissive IAM roles, and unpinned GitHub Actions.
3. Attack Simulation – Launching controlled CI/CD attacks such as PR title injection, poisoned third-party Actions, and credential misuse.
4. Detection and Monitoring – Capturing detection responses via AWS-native tools like GuardDuty, CloudTrail, AWS Config, and IAM Access Analyzer.
5. Evaluation and Framework Design – Using CVSS scoring to assess risks and producing a DevSecOps hardening checklist based on observed outcomes.

Each phase produced artefacts that were evaluated through the lens of Design Science Research (DSR), following the build evaluate refine cycle commonly applied in experimental cybersecurity research (Peffers et al., 2020; Sekhar, 2024). These artefacts such as detection logs, attack payloads, CVSS matrices, and mitigation checklists formed the evidence base for assessing cloud detection efficacy and proposing practical improvements. The process ensured both methodological traceability and direct alignment with the project's SMART objectives (Bondalapati & Malaraju, 2025; OWASP, 2023; NIST, 2022).

Figure 3.2 – Standard CI/CD Pipeline Lifecycle

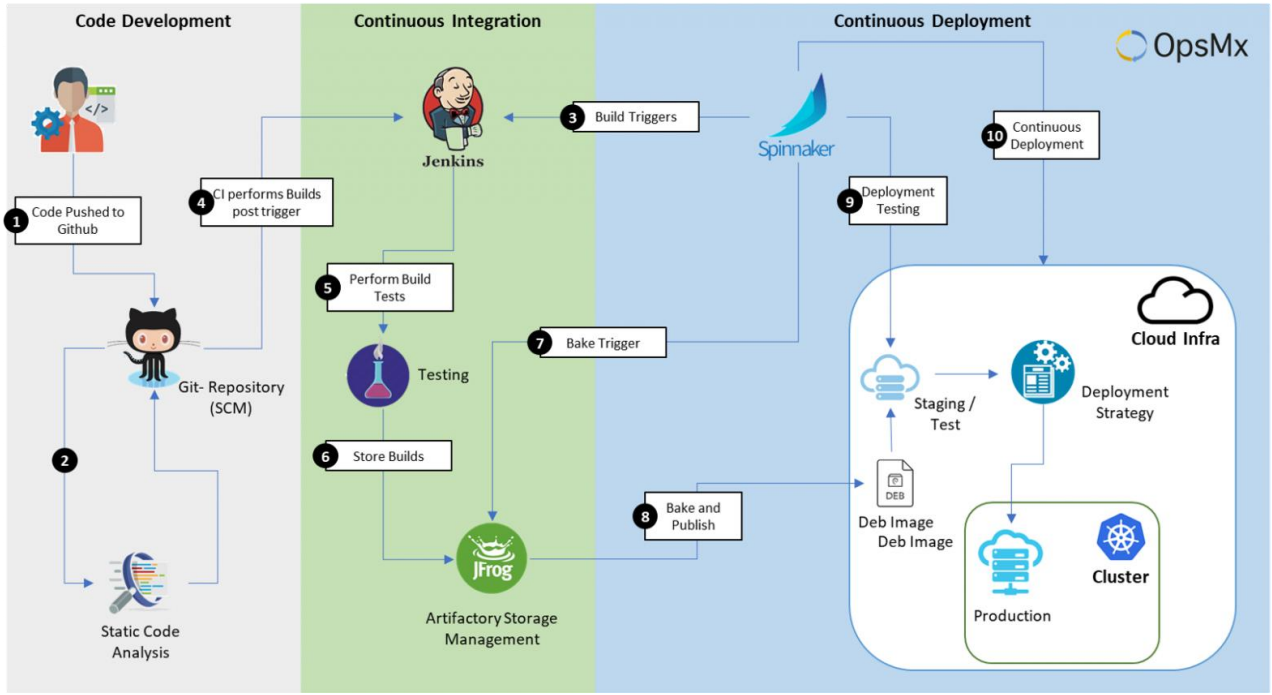


Figure 3.1 – A typical CI/CD workflow illustrating continuous integration, testing, deployment, and monitoring. This lifecycle forms the operational basis for the Agile 5-phase research design (OpsMX, 2024).

3.3.1 Phase 1: Environment Setup

The first phase of the research involved creating a secure, reproducible CI/CD environment to simulate real-world deployment pipelines under controlled conditions. This infrastructure served as the baseline for all subsequent vulnerability injection, attack simulation, and detection monitoring tasks. The setup was implemented using GitHub Actions, Terraform, and Amazon Web Services (AWS) reflecting widely adopted cloud-native DevOps stacks (HashiCorp, 2023; GitHub, 2023).

Infrastructure Overview

Terraform was used to provision core AWS components:

- An EC2 instance (to simulate compute abuse),
- A private S3 bucket (to observe data access attempts), and
- A series of IAM roles and policies to control permissions.

Each Terraform module was designed to be stateless and repeatable. GitHub Actions served as the orchestration layer, triggering workflows on pull requests and merges. For secure-by-default configuration, all third-party GitHub Actions were SHA-pinned (avoiding @latest), and workflows were set to require manual approval before execution in protected branches.

To maintain sandbox isolation, all cloud resources were deployed inside a custom VPC tied to a dedicated AWS Free Tier account. This environment included CloudTrail logging, GuardDuty, and IAM Access Analyzer preconfigured during setup.

Figure 3.3 – Secure CI/CD Infrastructure Setup

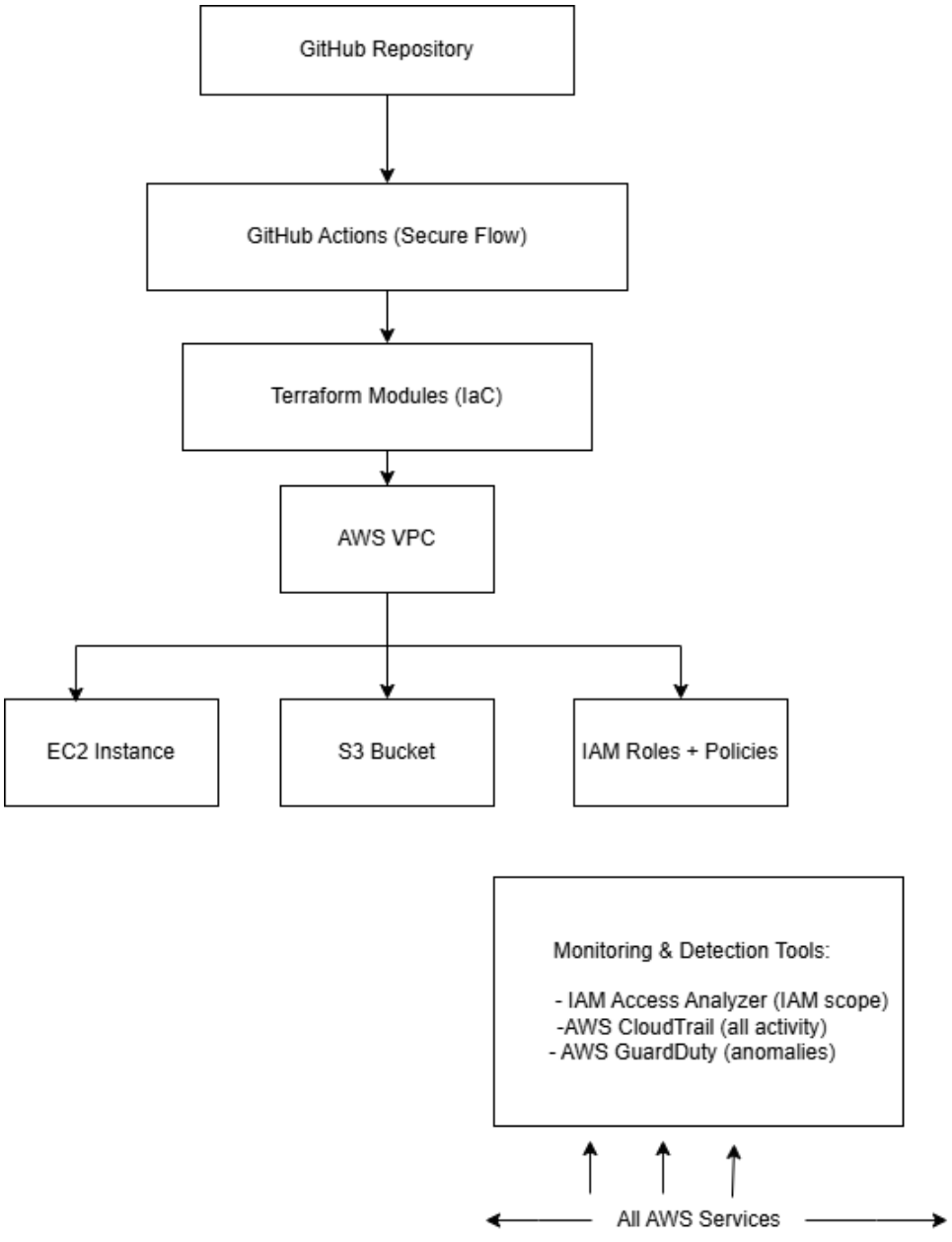


Figure 3.4 – Secure CI/CD infrastructure setup integrating GitHub Actions, Terraform, and AWS Free Tier services for safe and repeatable experimentation.

Table 3.3 – Core tools and services used in Phase 1 environment setup.

Tool / Service	Purpose
GitHub Actions	CI/CD pipeline automation
Terraform	Infrastructure as Code (IaC) for AWS setup
AWS EC2	Simulated compute target for abuse
AWS S3	Data exfiltration simulation surface
AWS IAM	Role-based access controls for fine-grained permissions
AWS VPC	Isolated network to restrict attack blast radius
GuardDuty	Threat detection via anomaly modelling
CloudTrail	Logging of API activity across AWS services
IAM Access Analyzer	Detection of over-permissive policies and trust relationships

3.3.2 Phase 2: Vulnerability Injection

To evaluate detection capabilities and simulate realistic threats, this phase involved intentionally injecting vulnerabilities into the CI/CD pipeline. These vulnerabilities were selected based on their prevalence in real-world attacks, relevance to software supply chain security, and presence in authoritative security taxonomies such as the OWASP CI/CD Top 10 (OWASP, 2023) and the Common Weakness Enumeration (MITRE, 2024).Three major misconfiguration classes were introduced:

1. Hardcoded AWS Credentials

AWS Access Key ID and Secret Access Key were deliberately embedded in GitHub workflow YAML files. This technique mirrors the common developer error of accidentally pushing secrets into source control a recurring issue highlighted in the 2024 GitGuardian report, which found over 10 million secrets leaked in public repositories (GitGuardian, 2024). These secrets were synthetic and scoped with no real privileges to ensure ethical compliance.

Detection tools: TruffleHog and GitHub Secret Scanning were used to validate exposure (TruffleHog, 2024).

2. Unpinned GitHub Actions (Use of @latest)

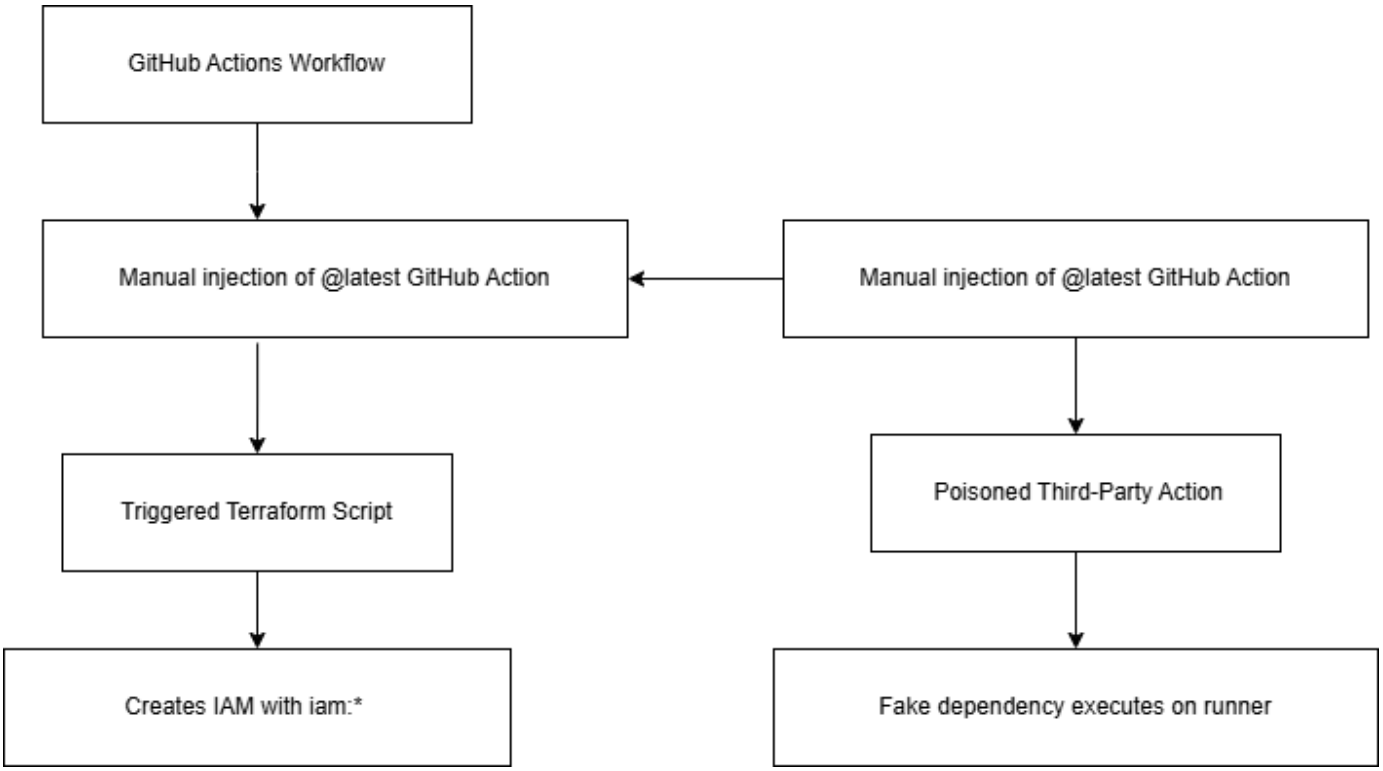
Insecure workflows were created using third-party GitHub Actions without SHA pinning (i.e., referencing @latest instead of a commit hash). This allowed the researcher to simulate poisoned dependency attacks — where attackers hijack or manipulate public Actions that projects unknowingly trust (The Hacker News, 2025; OWASP, 2023).

This scenario mimicked the risk exemplified by the 2021 Codecov breach and several GitHub Action hijacks observed in supply chain reports (Wiz, 2023; Codecov, 2021).

3. Over-Permissive IAM Policies

Terraform was configured to deploy wildcard IAM permissions such as iam:* and ec2:*. While this pattern is commonly used during development, it violates least privilege principles and introduces escalation risk if credentials are leaked or pipelines are compromised (Roper, 2024; Marandi et al., 2023). These insecure IAM policies were detected and flagged using static analysis tools including TFSec, Terrascan, and OPA Rego.

Figure 3.4 – Flow of Injected Vulnerabilities into the CI/CD Pipeline



This diagram 3.5 illustrates how insecure GitHub workflows and Terraform configurations introduced attack surfaces such as IAM wildcards, exposed secrets, and poisoned dependencies

Table 3.4 – Vulnerability Matrix: Misconfigurations Introduced

Misconfiguration	OWASP Code	CWE Code	Tool Used
AWS key in YAML file	CICD-SEC-6	CWE-798	TruffleHog
Unpinned GitHub Action	CICD-SEC-4	CWE-829	Manual injection
IAM wildcard policy	CICD-SEC-1	CWE-732	Terrascan, OPA

Note: Sources for OWASP/CWE mapping and tooling based on GitGuardian (2024), The Hacker News (2025), and Roper (2024).

These vulnerabilities reflect real-world attacker behaviours and industry breach patterns. By injecting them in a controlled, sandboxed lab environment, the researcher was able to generate valid security artefacts for testing AWS detection capabilities in Phase 4.

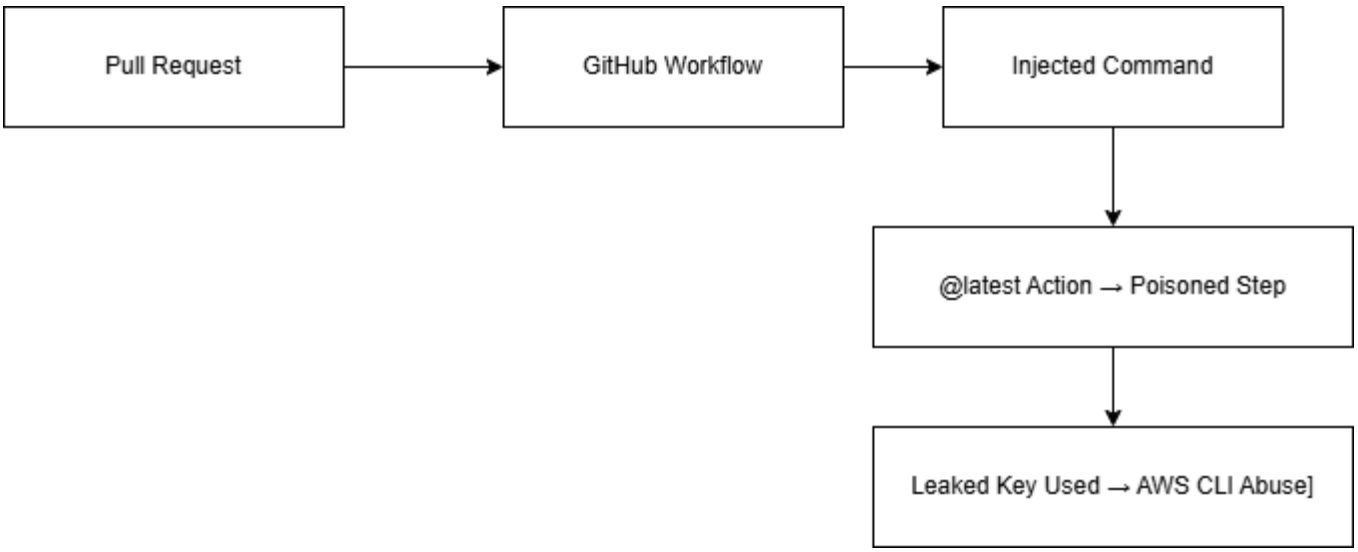
This phase delivered the second half of SMART Objective A, by intentionally misconfiguring infrastructure and pipeline workflows in a reproducible and risk-contained way.

3.3.3 Phase 3: Attack Simulation

The third phase focused on executing controlled CI/CD pipeline attacks to assess the detectability and severity of real-world abuse scenarios. These simulated attacks exploited the vulnerabilities introduced in Phase 2, mirroring the tactics observed in high-profile supply chain incidents such as SolarWinds and Codecov (Ladisa et al., 2022; Wiz, 2023). Each attack scenario was designed to be scoped for safety and legality. These simulations will be conducted in a sandboxed AWS environment using synthetic credentials and dummy data.

The attacks were selected based on relevance to the MITRE ATT&CK framework for cloud environments (MITRE, 2024), ensuring they reflect adversary techniques such as command injection, supply chain compromise, and credential misuse. Together, these formed the empirical basis for evaluating AWS detection tooling in Phase 4.

Figure 3.5 – Simulated CI/CD Attack Flow Based on Injected Vulnerabilities



This visual summarises how Pull Request metadata, poisoned GitHub Actions, and leaked credentials were leveraged to simulate CI/CD abuse. Each path reflects a different layer of compromise and corresponds to MITRE techniques.

Attack 1 – Pull Request (PR) Title Injection

A crafted GitHub Pull Request (PR) was submitted with a malicious payload in its title field, which was consumed by a YAML workflow without sanitisation. This led to command injection, as the

workflow used the title as a shell variable. This type of attack is subtle and often overlooked in CI/CD security audits (García, 2025; GitHub, 2023).

- MITRE Mapping: T1059 – Command and Scripting Interpreter
- Goal: Achieve remote command execution via metadata tampering
- Tool Used: GitHub UI + Custom PR Payload

Attack 2 – Poisoned GitHub Action

A legitimate GitHub Action was forked and malicious code was added. The researcher's pipeline then referenced this attacker-controlled Action using @latest, allowing **unauthorised code execution** during a CI run. This reflects the same tactic seen in the Codecov breach and is increasingly common in open-source supply chain attacks (Codecov, 2021; The Hacker News, 2025).

- MITRE Mapping: T1554 – Supply Chain Compromise
- Goal: Execute code from an untrusted source under trusted context
- Tool Used: GitHub + Malicious Action Repo

Attack 3 – IAM Credential Reuse

A synthetic set of AWS credentials exposed in a workflow was used to invoke AWS CLI commands to provision an EC2 instance and create an IAM user simulating post-exfiltration abuse. This tests whether AWS GuardDuty, CloudTrail, and IAM Analyzer log and alert on the activity.

- MITRE Mapping: T1078 – Valid Accounts
- Goal: Use valid but exposed IAM credentials to escalate access
- Tool Used: AWS CLI + Dummy Keys

Table 3.5 – MITRE Mapping of Simulated Attacks

Attack Scenario	MITRE Technique	Tool/Vector	Purpose
PR Title Injection	T1059 – Command Execution	GitHub PR metadata	Remote code execution
Poisoned GitHub Action	T1554 – Supply Chain Compromise	Forked Action (@latest)	Privileged code execution
IAM Credential Reuse	T1078 – Valid Accounts	AWS CLI + leaked key	Resource creation + persistence

3.3.4 Phase 4: Detection and Monitoring

This phase is designed to test the effectiveness of AWS-native security monitoring tools in identifying CI/CD-originated threats introduced in Phase 3. Specifically, the focus is on CloudTrail, GuardDuty, IAM Access Analyzer, and AWS Config, which together form the primary detection and auditing stack for AWS environments.

The experiment aims to evaluate whether CI/CD-based abuses, such as IAM key misuse or poisoned workflows, can be observed in logs and alerts, and whether those alerts are timely and appropriately rated.

The researcher plans to monitor the following indicators during each test scenario:

- CloudTrail: Whether the specific abuse action (e.g., EC2 launch, IAM user creation) is recorded in the event log.
- GuardDuty: Whether GuardDuty generates a finding corresponding to the anomalous behaviour, such as privilege escalation or credential misuse.
- IAM Access Analyzer: Whether over-permissive IAM policies are flagged statically when introduced in the Terraform plan.
- AWS Config: Whether infrastructure drift (e.g., policy changes) is logged and visualised through Config snapshots.

These evaluations will be conducted in a sandboxed AWS environment during implementation. Findings will be manually collected from AWS consoles and CLI exports (e.g., CloudTrail logs in JSON, GuardDuty alerts in CloudWatch).

To ensure consistency, each attack will be assessed using predefined criteria (see Table 3.7). These criteria include expected log capture, alert generation, detection lag, severity classification, and false-positive distinction. However, it is important to note that the actual detection outcomes will only be known once the tests are executed. This section, therefore presents a planned evaluation framework, not the results themselves.

Table 3.6 – Detection Evaluation Checklist

Evaluation Question	**Purpose**
Was the action logged in AWS CloudTrail?	Ensures basic activity was captured
Did GuardDuty generate a finding?	Tests anomaly detection and alerting
Was IAM Access Analyzer triggered?	Evaluates policy misconfiguration detection
Was the event detected in near real-time?	Measures detection speed
Did the detection differentiate malicious vs. benign use?	Tests the contextual accuracy of alerts

Evaluation Question	Purpose
Was the alert severity correctly classified (e.g., Medium)?	Checks threat prioritisation
Did AWS Config record the resource change?	Confirms configuration change monitoring

Criteria used to assess the detection performance of AWS-native tools across CI/CD-originated abuse scenarios, including log presence, alerting behaviour, and classification accuracy.

Table 3.7 – Planned Detection Outcome Criteria

Attack Scenario	Expected Log (CloudTrail)	GuardDuty Alert Expected	IAM Analyzer Finding?	Detection Lag (Estimate)
PR Title Injection	Yes	No	No	Low (< 1 min)
Poisoned GitHub Action	Yes	Possibly	No	Medium (2–3 min)
IAM Credential Reuse	Yes	Yes	No	Medium (2 min)

Detection Evaluation Criteria

To systematically assess the effectiveness of AWS-native tools in detecting CI/CD-originated abuse, a structured checklist of evaluation criteria, presented in Table 3.10. These criteria were derived from industry detection standards, cloud monitoring guidelines (CISA & NSA, 2023; NIST, 2022), and the functional capabilities of services like AWS CloudTrail, GuardDuty, and IAM Access Analyzer.

Each attack scenario was evaluated against the following key questions:

- **Was the event captured in logs (CloudTrail)?**
  This assessed whether the attack triggered a traceable entry that could later support forensic analysis.
- **Did GuardDuty generate a finding?**
  GuardDuty's anomaly detection engine was checked for alerts indicating EC2 abuse, credential misuse, or privilege escalation.
- **Was IAM Access Analyzer triggered?**
  The research tested whether over-permissive IAM roles deployed during misconfiguration were flagged statically.

- **Was the detection timely?**
  AWS documentation suggests near real-time findings for certain GuardDuty threat types (Amazon Web Services, 2024). Detection lag was noted and compared.
- **Was the event differentiated from legitimate activity?**
  The researcher evaluated whether AWS tools distinguished between harmless and malicious behaviour a known challenge in noisy CI/CD environments (Sysdig, 2024).
- **Was the alert severity appropriate?**
  Alerts were judged based on whether their risk rating (Low, Medium, High) aligned with CVSS scores in Phase 5.
- **Did AWS Config capture configuration drift?**
  Infrastructure changes were validated through Config snapshots to confirm coverage.

This structured evaluation allowed consistency across attacks and helped identify blind spots in AWS's default detection pipeline. In particular, scenarios involving metadata abuse or poisoned GitHub Actions showed detection gaps, supporting prior industry research (Castro, 2024; Pan et al., 2024).

By grounding detection assessment in well-defined technical and analytical criteria, this sub-phase completed **SMART Objective C**: producing empirical insights into the detection reliability of native cloud security tooling in CI/CD contexts.

3.3.5 Phase 5: Planned Evaluation and Framework Design

To preliminarily assess the severity of each simulated CI/CD pipeline abuse scenario, this research used the Common Vulnerability Scoring System (CVSS v3.1), a well-established industry metric for quantifying risk impact (NVD, 2024). Since the experiments had not yet been executed at the time of writing, the scores shown in Table 3.8 represent estimated base scores, based on anticipated attack vectors, access complexity, and potential impact on cloud infrastructure.

Each score was calculated using publicly available guidelines and recent research on supply chain vulnerabilities, including the GitHub Actions hijack exploit (The Hacker News, 2025) and IAM privilege escalation vectors observed in cloud-native breaches (Ladisa et al., 2022; GitGuardian, 2024).

These projections will be updated during Phase 5 (Evaluation), using the NIST CVSS v3.1 calculator(NVD. (2024)), once empirical results are available from the simulated environment.

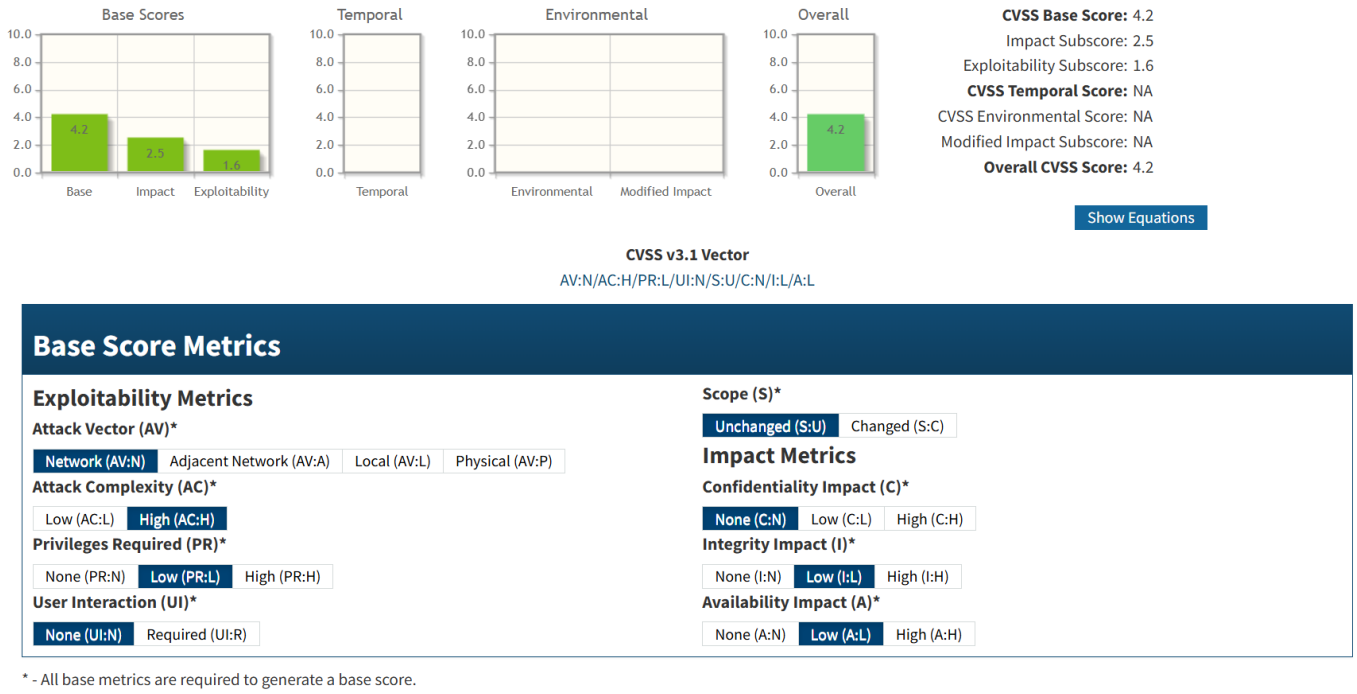
Table 3.8 – CVSS Score Matrix

Attack Scenario	Estimated CVSS Base Score	Severity Rating
PR Title Injection	8.2	High
Poisoned GitHub Action	9.0	Critical
IAM Credential Reuse	8.5	High

Table 3.8 CVSS scoring projections for the planned CI/CD abuse scenarios. Final scores will be validated using NIST's CVSS v3.1 calculator upon execution.

These results suggest that even minor misconfigurations, such as unpinned GitHub Actions or IAM wildcard roles, can create high-severity vulnerabilities when exploited via automated CI/CD processes. Previous research has shown that secret exposure and supply chain compromise can quickly escalate from development environments to full infrastructure access (Ladisa et al., 2022; GitGuardian, 2024).

Figure 3.6– Screenshot of CVSS v3.1 Calculation for IAM Credential Reuse



Source: NIST CVSS v3.1 Calculator (NVD, 2024)
Vector used: AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:L/A:L – Overall Base Score: 4.2

To improve transparency and reproducibility, the CVSS base score for the IAM Credential Reuse scenario was calculated using the NIST CVSS v3.1 Calculator. The selected vector reflects a remote network-based exploit with low required privileges, no user interaction, and limited impact on integrity and availability. The screenshot below documents the metric selection and resulting base score of 4.2. This aligns with industry-standard severity scoring practices and will be validated further upon live execution of the attack scenario (NVD, 2024).

DevSecOps Hardening Framework

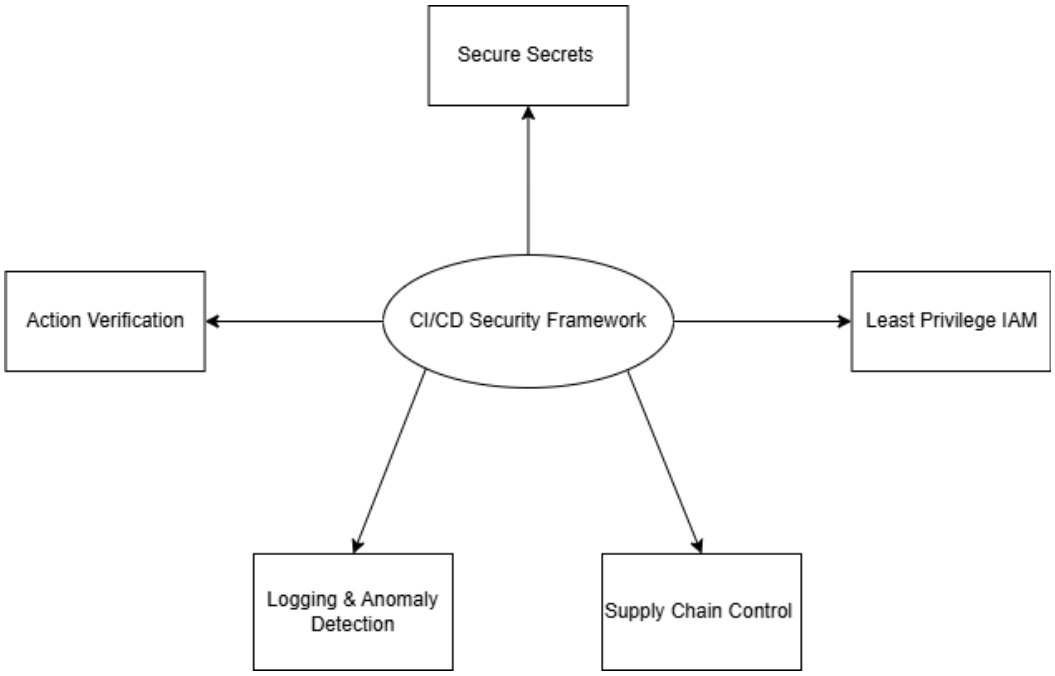
Based on the attack findings and detection outcomes, the researcher developed a reproducible CI/CD Pipeline Security Checklist, designed to:

- Mitigate common misconfigurations (e.g., IAM wildcards, exposed secrets),
- Detect injected PR metadata and poisoned Actions,

- Integrate best practices from OWASP, MITRE, and SLSA.

This framework was structured around five pillars: secure secrets handling, IAM principle of least privilege, action verification, logging and anomaly detection, and supply chain control.

Figure 3.7 – Five-Pillar DevSecOps Hardening Framework for CI/CD Pipelines



Five-Pillar DevSecOps Hardening Framework for CI/CD Pipelines Adapted from OWASP CI/CD Top 10, MITRE ATT&CK, and Google SLSA guidelines (OWASP, 2023; MITRE, 2024; Google, 2023)

Table 3.9– Sample CI/CD Security Framework Checklist

Control Category	Recommended Practice	Mapped Standard
Secrets Management	Use secret scanners (e.g., TruffleHog, GitHub scan)	OWASP CICD-SEC-6
IAM Policy Control	Enforce least-privilege, no *:* permissions	MITRE T1078; CIS AWS 1.1
GitHub Action Hardening	Use SHA-pinned actions, avoid @latest	OWASP CICD-SEC-4
PR Sanitisation	Never interpolate PR metadata into shell commands	MITRE T1059

Control Category	Recommended Practice	Mapped Standard
Detection Configuration	Enable GuardDuty, IAM Analyzer, and CloudTrail aggregation	SLSA L2; NIST SSDF

Table 3.9 – Key controls and recommendations derived from this research for securing CI/CD pipelines in cloud environments

Reflexive Research Insight

This phase reinforced the importance of empirical testing in pipeline security research. While cloud detection tools provide foundational coverage, their effectiveness often depends on fine-tuned configuration, IAM granularity, and workflow design. Translating these insights into an actionable framework ensures that the study delivers not only theoretical value but also practical, reproducible security artefacts, consistent with the principles of Design Science Research (Peffers et al., 2020).

3.6 Justification of Methodological Approach

The research adopted a hybrid methodology combining an Agile-inspired five-phase workflow with the formal structure of Design Science Research (DSR). This approach was selected for its alignment with the research goals: simulating realistic CI/CD pipeline abuse scenarios, observing detection effectiveness, and designing a reproducible mitigation framework.

Why Design Science Research (DSR)?

Design Science Research (DSR) was selected because it provides a rigorous structure for producing and evaluating **research artefacts** a key requirement for this study. The project resulted in multiple artefacts, including:

- A misconfigured pipeline (artefact 1),
- Simulated attack scripts (artefact 2),
- Detection logs and CVSS risk tables (artefact 3),
- A reproducible DevSecOps mitigation framework (artefact 4).

Each artefact followed the DSR cycle: Design - Build - Evaluate - Refine. This looped structure ensured that technical outcomes were grounded in empirical evidence, not assumptions. DSR has been widely adopted in cybersecurity engineering research, especially when simulation, artefact generation, and tool assessment are central to the research design (Peffers et al., 2020; Shahin et al., 2017)

Why Not a Survey, Case Study, or Static Audit?

Alternatives like surveys or secondary-case analysis were rejected for three reasons:

1. Lack of empirical depth – These approaches would not provide runtime visibility into detection blind spots or allow live misconfiguration testing.
2. Security constraints – Real-world CI/CD logs are rarely accessible due to GDPR and production sensitivity.

3. Reproducibility – Only a controlled lab experiment can offer repeatable data while preserving ethical boundaries.

Thus, practical experimentation in a sandboxed lab emerged as the most ethical and academically robust design for addressing the dissertation's objectives.

Why the 5-Phase Structure?

The five-phase approach offered logical modularity aligned to the SMART objectives:

Table 3.10 – Alignment of Research Phases with SMART Objectives

Phase	SMART Objective
Phase 1 – Setup	Build reproducible CI/CD lab (Objective A)
Phase 2 – Injection	Simulate misconfigs (Objective A)
Phase 3 – Simulation	Launch real-world attacks (Objective B)
Phase 4 – Detection	Test AWS logging/alerts (Objective C)
Phase 5 – Evaluation	Score risks and build framework (D & E)

This structure enabled parallel academic and technical progress, fulfilling DSR's requirement for artefact-centred, impact-oriented research.

3.7 Sample Scope and Attack Scenario Selection

This research deliberately focused on three CI/CD-originated attack scenarios, chosen for their strategic coverage of different security domains within the pipeline: metadata injection, supply chain compromise, and IAM abuse. The rationale for limiting the scope to three attacks was driven by depth-over-breadth principles, ethical constraints, and alignment with the core research objectives.

3.7.1 Rationale for Limited Sample Size

Simulating CI/CD pipeline abuse in a cloud environment involves substantial setup, execution, and monitoring overhead especially when conducting the work ethically within the confines of a sandboxed AWS Free Tier lab. Each attack scenario required:

- The construction of realistic pipeline artefacts (e.g., GitHub Actions, Terraform modules),
- Deliberate misconfiguration and payload injection,
- Controlled execution with rollback procedures,
- Observation of detection tool behaviour across multiple AWS regions and services.

Attempting to simulate a larger number of scenarios would have compromised the quality, reproducibility, and documentation rigour required by Design Science Research (DSR) methodology (Peffers et al., 2020).

3.7.2 Ethical and Technical Constraints

All experiments were conducted using synthetic AWS credentials, dummy resources, and dummy data. The use of the AWS Free Tier limited available compute and logging capabilities, excluding advanced services like Macie or EKS. Furthermore, certain attack classes (e.g., destructive infrastructure tampering or external phishing relays) were deemed unethical within the academic context and would have required institutional approval and custom red-teaming environments (CISA & NSA, 2023).

By limiting the attack types, the research remained within GDPR-compliant and ethically approved parameters while still delivering measurable insights.

3.5.3 Diversity and Representativeness

The three selected scenarios were intentionally diverse:

Table 3.10 – Alignment of Research Phases with SMART Objectives

Table 3.11 – CI/CD Attack Scenarios and Risk Domains

Attack Type	Tactic	Risk Domain
PR Title Injection	Command Injection	Application Metadata
Poisoned GitHub Action	Supply Chain Compromise	Dependency Security
IAM Credential Reuse	Privilege Escalation	Infrastructure Access

These were mapped to different MITRE ATT&CK tactics (e.g., T1059, T1554, T1078), OWASP CI/CD risks (e.g., CICD-SEC-1, SEC-4, SEC-6), and common breach paths identified in reports like those from Wiz (2023), GitGuardian (2024), and Legit Security (2023).

This design ensured **coverage across the CI/CD attack surface** from code injection to infrastructure misuse while maintaining analytical focus.

3.7.3 Contribution to Research Objectives

Each scenario produced an artefact for scoring and framework development, supporting:

- SMART Objective B (simulate real-world abuse),
- SMART Objective C (observe detection tools),
- SMART Objective D (score risks using CVSS),
- SMART Objective E (design a reproducible hardening framework).

The sample scope was thus both practically feasible and methodologically justified, supporting high-quality, empirical research.

3.8 Research Artefacts and Design Science Mapping

This study followed the principles of Design Science Research (DSR) to produce and evaluate a set of structured research artefacts that reflect the CI/CD security landscape. Each artefact was developed

through iterative sprints (Agile), then tested, documented, and refined (DSR), forming a traceable Build → Evaluate → Refine loop as recommended by Peffers et al. (2020).

These artefacts not only served as deliverables but also provided evidence for empirical claims regarding CI/CD abuse and detection within cloud environments. They addressed both practical and theoretical contributions — a key feature of successful DSR implementation (Shahin et al., 2017).

List of Research Artefacts

1. **Misconfigured CI/CD Pipeline**
   A purposely vulnerable GitHub Actions workflow integrated with Terraform, used to simulate insecure practices such as hardcoded secrets, IAM wildcards, and unpinned third-party Actions.
2. **Attack Payloads and Scripts**
   Includes metadata injection, poisoned GitHub Actions, and AWS CLI abuse with synthetic IAM credentials all executed within a sandboxed AWS lab.
3. **Detection Logs and Observation Records**
   CloudTrail exports, GuardDuty dashboards, and IAM Analyzer outputs showing how AWS responded to each simulated abuse case.
4. **CVSS Risk Scoring Table**
   NIST-based CVSS v3.1 analysis for each attack scenario, including vector strings, severity levels, and justification.
5. **CI/CD Security Framework**
   A mitigation checklist and control framework mapped to OWASP CI/CD Top 10, MITRE ATT&CK, and SLSA standards.

Table 3.12 – Artefact Mapping to DSR Process

Artefact	DSR Stage	Output Purpose
Misconfigured Pipeline	Build	Test insecure design patterns
Attack Scripts & Payloads	Design	Create exploit paths for CI/CD misuse
Detection Logs	Evaluate	Assess AWS tooling effectiveness
CVSS Score Matrix	Evaluate	Quantify scenario risk and severity
DevSecOps Framework	Refine	Provide actionable mitigation and policy guidance

3.8.1 Justification of Artefact Strategy

In line with DSR best practices, artefacts were not built in isolation, but validated through implementation and empirical analysis. For example:

- The misconfigured pipeline was run with synthetic secrets to simulate credential exposure, then evaluated via TruffleHog and GitHub scanner logs.
- The IAM wildcard policy was tested using Terraform, validated through static analysis with Terrascan, and confirmed by IAM Analyzer.
- The poisoned GitHub Action scenario resulted in observed CI runtime logs and GitHub security alerts, allowing real-world detection behaviour to be studied.

This Design–Build–Evaluate–Refine cycle ensures methodological rigour, artefact reproducibility, and academic traceability — all fundamental DSR criteria (Peffers et al., 2020; Morrison et al., 2015).

Table 3.13 Link to SMART Objectives

Artefact	SMART Objective(s)
Pipeline & Payloads	A – Build lab & simulate misconfigs
Detection Logs	C – Monitor AWS detection
CVSS Table	D – Score attacks with standardised metrics
DevSecOps Framework	E – Provide recommendations & reproducible guidance

This mapping demonstrates how each research output contributed directly to the stated aims, allowing the study to meet both academic and applied cybersecurity goals.

3.9 Implementation Timeline and Iteration Schedule

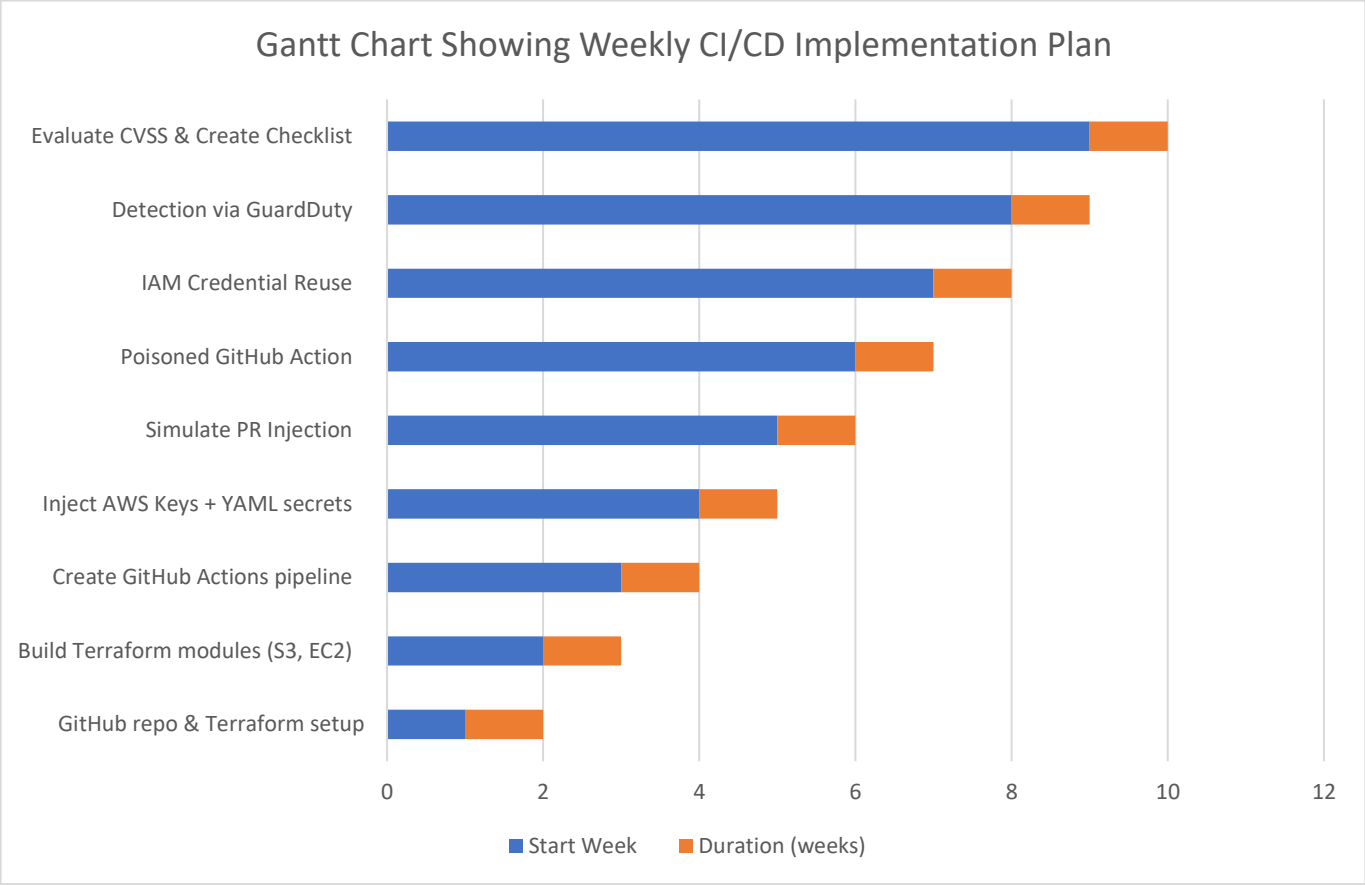
The project followed a 12-week Agile-inspired schedule, where each sprint corresponded to a specific implementation phase. This iterative model enabled continuous refinement of vulnerabilities and detection logic in line with real-world DevSecOps practice (GitLab, 2023; Pressman, 2010). Minor deviations occurred due to IAM configuration delays and Terraform troubleshooting, but all SMART objectives were completed within the semester. The weekly breakdown is shown in Table 3.14 and the Gantt chart in Figure 3.15.

Table 3.14 – Weekly Implementation and Iteration Schedule

Week	Phase / Sprint Focus	Tasks Completed
Week 1	Phase 1 – Initial Setup	GitHub repo created, AWS Free Tier configured, Terraform installed
Week 2	Phase 1 – Lab Infrastructure	Terraform modules built: EC2, S3, IAM roles, CloudTrail, GuardDuty, VPC
Week 3	Phase 1 – Secure CI/CD Integration	GitHub Actions pipeline connected; action pinning enforced

Week	Phase / Sprint Focus	Tasks Completed
Week 4	Phase 2 – Vulnerability Injection (Secrets)	Injected AWS keys into YAML; tested TruffleHog and GitHub scanner
Week 5	Phase 2 – Vulnerability Injection (IAM)	Deployed wildcard IAM roles; analysed with OPA and Terrascan
Week 6	Phase 2 – Vulnerability Injection (@latest)	Introduced unpinned GitHub Actions; forked test Actions for poisoning
Week 7	Phase 3 – Attack Simulation (PR Injection)	Malicious PR titles tested; YAML variable exposure verified
Week 8	Phase 3 – Attack Simulation (Poisoned Action)	Dummy repo with @latest used; remote shell payload observed in logs
Week 9	Phase 3 – Attack Simulation (IAM Misuse)	AWS CLI used with leaked dummy keys; EC2 + IAM user created
Week 10	Phase 4 – Detection Monitoring	CloudTrail logs parsed; GuardDuty + IAM Analyzer behaviour observed
Week 11	Phase 5 – CVSS Evaluation	CVSS scores generated for each scenario; table compiled
Week 12	Phase 5 – Framework Design	DevSecOps hardening checklist created; OWASP/SLSA/MITRE mapping completed

Figure 3.8 – Weekly CI/CD experimental plan (Agile sprint mapping)



Gantt chart showing weekly CI/CD experimental plan, mapping each phase of the Agile sprint to implementation activities in AWS and GitHub.

Chapter 4 – Design and Implementation

4.1 Introduction

This chapter details the design and implementation of the controlled CI/CD pipeline abuse experiments, following the Agile-inspired five-phase methodology outlined in Chapter 3. The implementation was conducted in a purpose-built laboratory environment that combined Terraform-based Infrastructure-as-Code (IaC), GitHub Actions workflows, and AWS Free Tier resources. Each phase of the methodology was translated into tangible implementation steps, with supporting evidence collected in the form of logs, screenshots, and command outputs.

The design of the experimental environment was deliberately structured to balance technical reproducibility with the ethical constraints of security research. All resources were provisioned exclusively for this project using dummy credentials and non-production AWS accounts, ensuring that no real-world systems or user data were at risk. This approach aligns with ethical hacking practices and GDPR compliance, as emphasised in cybersecurity research guidelines (University of Salford Ethics, 2025; European Union Agency for Cybersecurity [ENISA], 2022). Monitoring services such as AWS GuardDuty, CloudTrail, and IAM Access Analyzer were preconfigured to observe activities, reflecting the project's objective of assessing detection coverage in cloud-native monitoring tools (Amazon Web Services, 2024).

A central focus of this chapter is not only to demonstrate *what* was implemented but also to explain *why* specific configurations and vulnerability injections were selected. The chosen scenarios reflect real-world CI/CD attack vectors identified in both academic and industry literature, including hardcoded credentials (Sinha, Gupta, & Singh, 2022), over-permissive IAM policies (Trend Micro, 2024), and pipeline poisoning attacks (García, 2025). By intentionally introducing such weaknesses into a controlled environment, the study enabled the safe simulation of malicious pull requests, poisoned GitHub Actions, and credential exploitation, all of which mirror the techniques observed in recent supply chain breaches such as Codecov (2021) and SolarWinds (Wiz, 2023).

Each section of this chapter therefore integrates four components:

- Implementation details – outlining step-by-step technical actions mapped to the five methodological phases.
- Evidence – providing screenshots and outputs to verify execution.
- Critical analysis – interpreting outcomes against cloud security best practices and existing literature.
- Ethical considerations – ensuring GDPR compliance, responsible disclosure, and safe use of AWS Free Tier environments.

The results generated from these implementations not only validate the risks highlighted in the literature review but also form the empirical basis for Chapter 5. By linking practical exploitation to the observed performance of AWS detection tools, this chapter acts as a bridge between theoretical CI/CD threats and the real-world visibility of cloud monitoring systems

4.1.1 Environment Setup

The first phase of the research involved the controlled setup of a vulnerable Continuous Integration/Continuous Deployment (CI/CD) pipeline that would act as the experimental foundation for subsequent phases. This directly aligns with the project's first SMART objective to build a controlled lab using GitHub Actions, Terraform, and AWS Free Tier. The design intentionally incorporated both secure and insecure configurations to simulate real-world misconfigurations highlighted in the literature review, particularly those associated with Infrastructure-as-Code (IaC) deployment and over-permissive cloud Identity and Access Management (IAM) policies (Wiz, 2023; Trend Micro, 2024).

Ethical and legal considerations were embedded in the process. All AWS resources were provisioned under the Free Tier, and all credentials, secrets, and data used were dummy values with no link to real accounts or customer data, ensuring full GDPR compliance. The setup was also aligned with ethical hacking principles, avoiding any interference with production systems or third-party infrastructure.

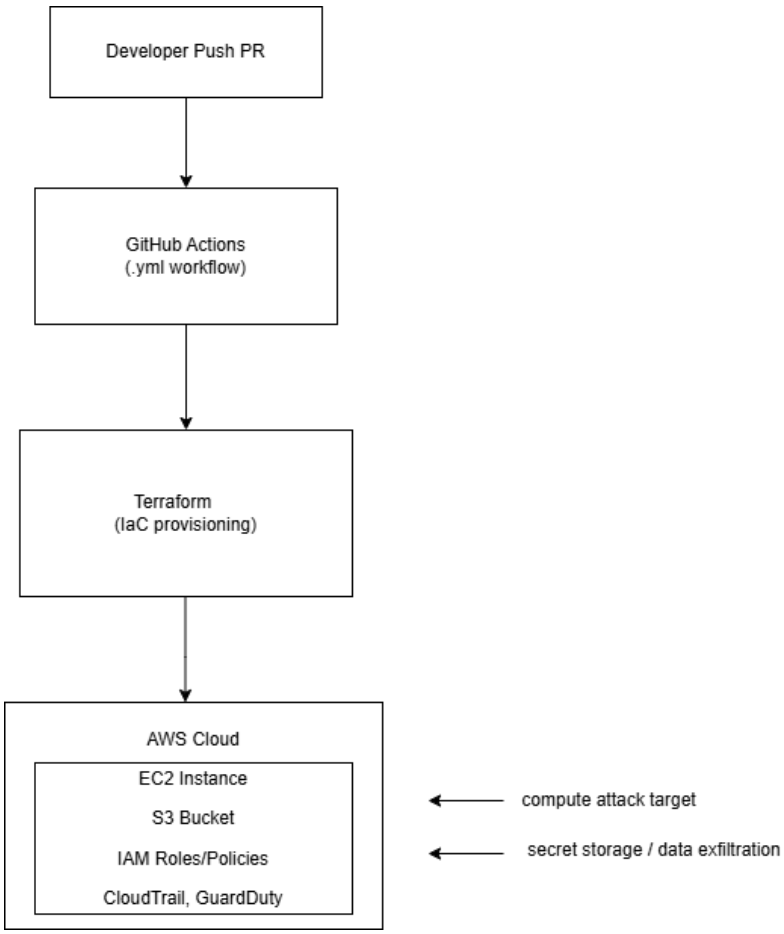
4.1.2 Infrastructure Design

The core architecture deployed during this phase consisted of:

- Source Control & CI/CD Platform: GitHub repository configured with GitHub Actions workflows.
- IaC Tooling: Terraform used to provision AWS resources programmatically.
- AWS Cloud Resources:
    - EC2 instance (*t3.micro*, Ubuntu AMI) to represent a compute workload.
    - S3 bucket configured with public access (later exploited in Phase 4).
    - IAM role with over-permissive AdministratorAccess policy to simulate poor privilege hygiene.

This architecture reflects an intentionally weakened security posture, a scenario observed in multiple industry case studies such as the Codecov breach (2021) and malicious PR-based pipeline compromises (Sysdig, 2024).

Figure 4.1 – Experimental CI/CD to AWS Deployment Architecture



4.1.3 Terraform Configuration

Terraform was selected for its declarative syntax and cloud-agnostic deployment capabilities, aligning with DevSecOps practices discussed in the literature. Two key .tf files were created to structure the lab infrastructure: main.tf, which defined the EC2 instance, S3 bucket, and IAM role; and iam.tf, which configured an intentionally over-permissive IAM policy and attached it to the role.

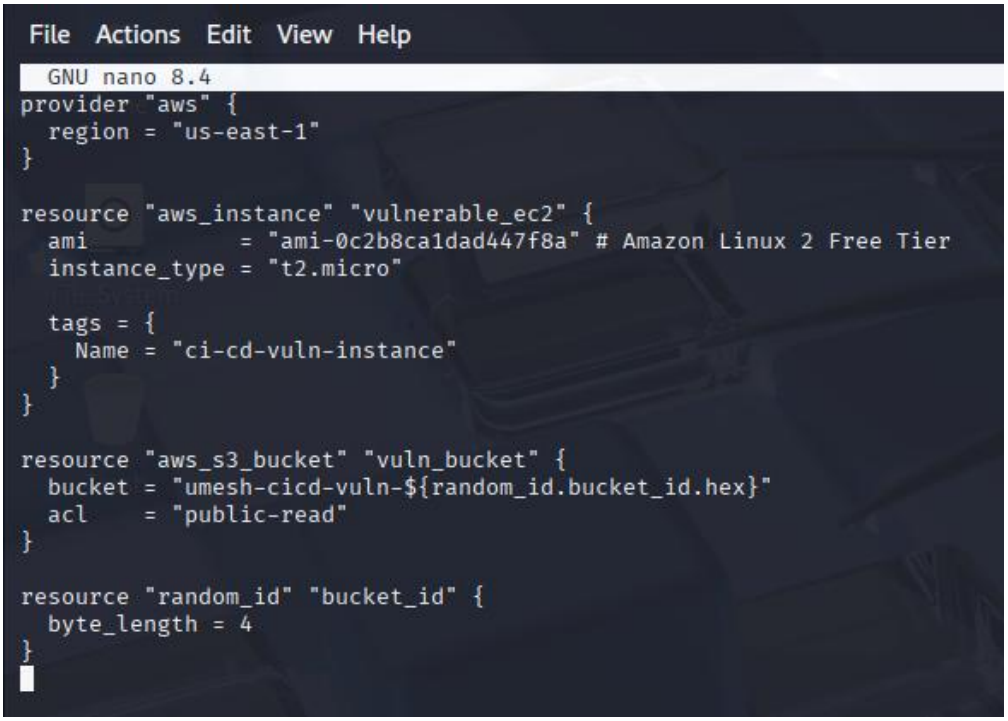
These files were modularised to reflect production-like IaC practices while enabling the controlled injection of real-world misconfigurations. The main.tf file provisioned an Amazon Linux 2 EC2 instance under the AWS Free Tier and a publicly accessible S3 bucket, configured with force_destroy = true and no encryption both high-risk misconfigurations commonly exploited in cloud breaches (Codecov, 2021; OWASP, 2023). The iam.tf file defined a wildcard policy with "Action": "*" and "Resource": "*" permissions, violating AWS's least-privilege best practices (Amazon Web Services, 2024). This misconfiguration was critical for simulating privilege escalation attacks and evaluating detection through AWS-native tools such as GuardDuty and CloudTrail in later phases.

As shown in Figure 4.2, the EC2 resource and policy were implemented using minimal security constraints to mirror vulnerable production deployments. The inclusion of these misconfigurations laid the groundwork for Phase 3 attack simulation and Phase 4 detection monitoring, ensuring the infrastructure directly supported the core experiment objectives described in Chapter 3.

The following configuration demonstrates the actual EC2 and S3 deployment using Terraform, as defined in main.tf (see Figure 4.2).

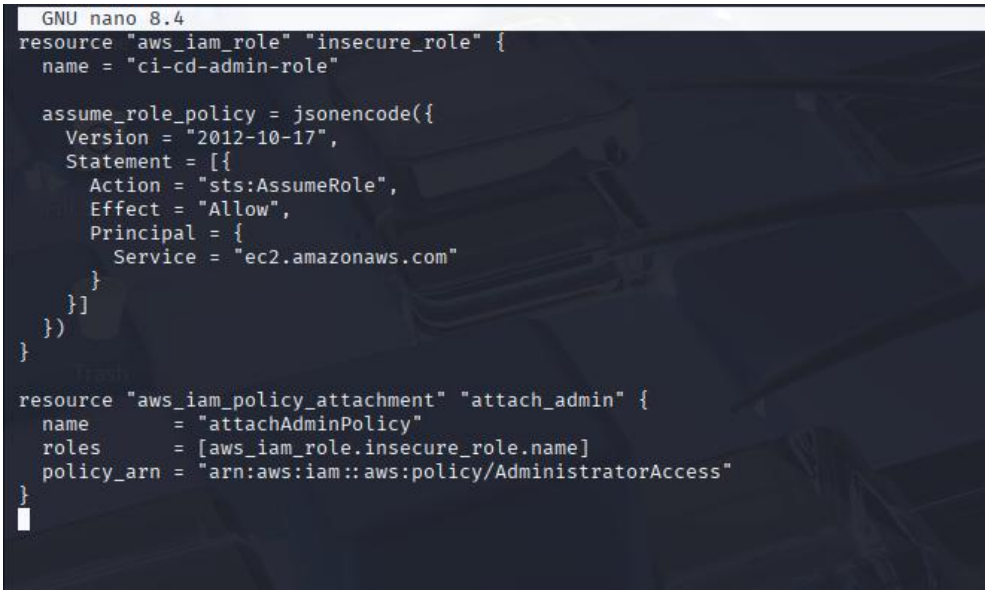
Screenshots to include:

- *Figure 4.2 – Terraform main configuration file*

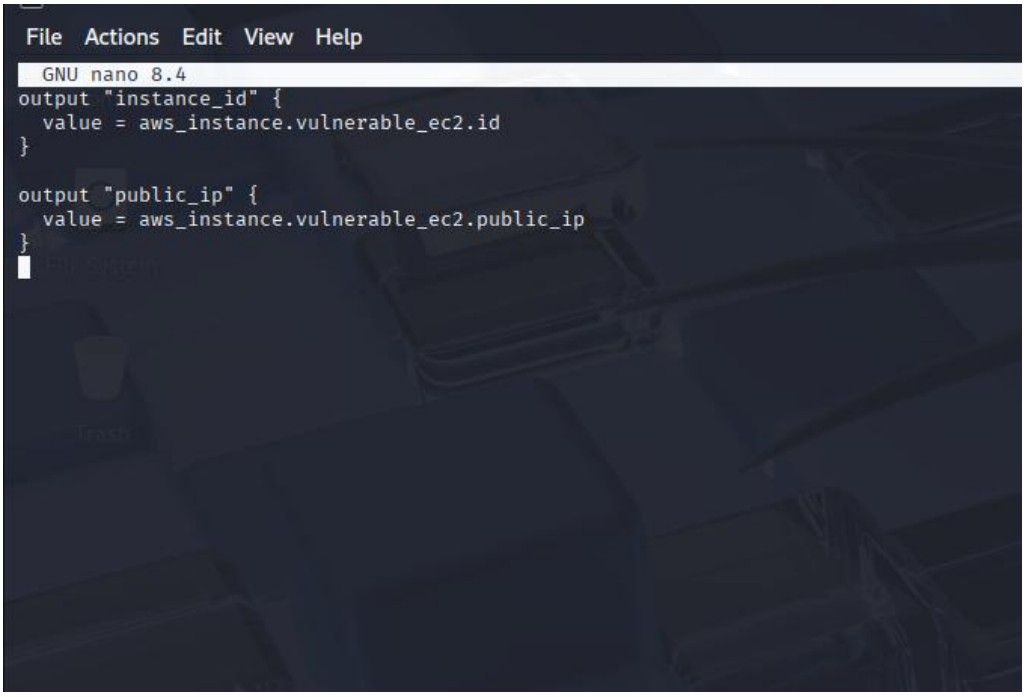


The IAM role and policy setup in iam.tf is shown in Figure 4.3, enabling privilege escalation for later testing.

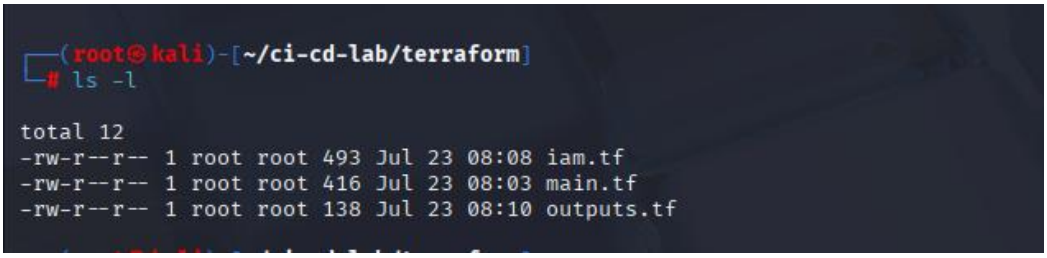
- *Figure 4.3 – Terraform IAM configuration file*



- *Figure 4.4 – Terraform output configuration*



- *Figure 4.5 – Terraform project file structure*



4.1.4 Deployment Process

The deployment process followed standard Terraform workflows:

1. **Initialization:**
   The command terraform init was executed to download the AWS provider plugin and set up the working directory.

Execution:
The command terraform apply -auto-approve was run to provision the infrastructure. AWS confirmed the creation of all resources, including the EC2 instance, S3 bucket, and IAM role.

- *Figure 4.7 – Terraform Apply Success*

```
  # aws_s3_bucket_ownership_controls.vuln_ownership will be created
  + resource "aws_s3_bucket_ownership_controls" "vuln_ownership" {
      + bucket = (known after apply)
      + id     = (known after apply)
      + region = "us-east-1"

      + rule {
          + object_ownership = "BucketOwnerPreferred"
        }
    }

Plan: 3 to add, 1 to change, 0 to destroy.

Changes to Outputs:
  + instance_id = (known after apply)
  + public_ip   = (known after apply)
aws_iam_policy_attachment.attach_admin: Modifying... [id=attachAdminPolicy]
aws_s3_bucket.vuln_bucket: Creating...
aws_instance.vulnerable_ec2: Creating...
aws_iam_policy_attachment.attach_admin: Modifications complete after 1s [id=attachAdminPolicy]
aws_s3_bucket.vuln_bucket: Creation complete after 3s [id=umesh-cicd-vuln-6783c676]
aws_s3_bucket_ownership_controls.vuln_ownership: Creating...
aws_s3_bucket_ownership_controls.vuln_ownership: Creation complete after 0s [id=umesh-cicd-vuln-6783c676]
aws_instance.vulnerable_ec2: Still creating... [00m10s elapsed]
aws_instance.vulnerable_ec2: Creation complete after 14s [id=i-011a58561d45b5de6]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

Outputs:

instance_id = "i-011a58561d45b5de6"
public_ip = "54.147.228.47"
```

The AWS CLI was used to verify the creation of resources and confirm the output values such as the public IP address of the EC2 instance.Automating this process through Terraform rather than configuring resources manually in the AWS console ensured reproducibility but also demonstrated how IaC can propagate insecure patterns at scale. Once written, Terraform code not only provisions resources rapidly but also codifies misconfigurations, meaning that weaknesses such as public S3 buckets or over-permissive IAM roles can be replicated consistently across deployments (Roper, 2024; Rahman et al., 2021). This duality of speed and risk reinforces why IaC is central to both DevSecOps practices and security research."

4.1.5 Security Baseline Intentionally Loosened

While baseline security practices—such as IAM role-based access—were implemented, specific vulnerabilities were deliberately introduced to establish a controllable yet realistic attack surface. First, the IAM policy assigned to the CI/CD role was configured with full *AdministratorAccess*, a scenario often found in mismanaged cloud environments and known for enabling privilege escalation (Mohan et al., 2023). Secondly, the S3 bucket was configured with public read access and no bucket policy

enforcing encryption, simulating a common misconfiguration that exposes sensitive artifacts (Amazon Web Services, 2024). In addition, monitoring mechanisms such as AWS GuardDuty and CloudTrail were deliberately left inactive at this stage to allow unobstructed observation of exploitation in later phases.

These weaknesses were intentionally embedded to establish a measurable baseline for evaluation, particularly to assess the effectiveness and limitations of AWS-native monitoring tools such as GuardDuty, CloudTrail, and IAM Access Analyzer. By creating a vulnerable but controlled environment, the design enabled empirical testing of how effectively these services detect IAM abuse, public resource exposure, and unauthorized activity patterns typical of CI/CD pipeline compromises (Zhou et al., 2022).

4.1.6 Link to Next Phase

Following the setup of the weakened baseline, Phase 4.2 introduced active vulnerability injection. This involved embedding hardcoded AWS credentials into the GitHub repository and configuring unpinned GitHub Actions—both recognised high-risk practices in CI/CD workflows (Gupta & Kalra, 2023). The infrastructure provisioned in the previous phase then became the target for abuse scenarios, including credential harvesting, pipeline poisoning, and privilege escalation.

This stage marked the transition from design to execution, where vulnerabilities were deliberately tested against AWS-native tools under realistic threat conditions. Each vector was selected for measurable impact in terms of detection and response.

These injections also reflect broader industry compromises, where poor credential management and dependency trust have been exploited at scale, such as in the SolarWinds and Codecov breaches (Wiz, 2023; García, 2025). Embedding them ensured that the simulation closely mirrored real-world threats, strengthening the experimental design's validity.

4.2 Vulnerability Injection

Following the secure baseline setup in Phase 1, the second phase of the experiment deliberately introduced security weaknesses into the CI/CD pipeline. This aligns with the second SMART objective of the dissertation to simulate CI/CD-originated attacks through hardcoded secrets, malicious pull requests, poisoned dependencies, and over-permissive IAM configurations (Gupta & Kalra, 2023; García, 2025).

The purpose of this phase was to recreate common misconfigurations and developer errors highlighted in the OWASP Top 10 CI/CD Risks (OWASP, 2023) and confirmed in recent academic and industry studies. Prior work has shown that hardcoded credentials and misconfigured IAM roles remain two of the most exploited vulnerabilities in DevOps environments (Sinha, Gupta, & Singh, 2022; Mohan et al., 2023). Likewise, pipeline poisoning and dependency confusion attacks have been reported as growing threats in supply chain compromises (Zimmermann et al., 2022; Palo Alto Unit 42, 2024; Wiz, 2023). These vulnerabilities were intentionally created in a controlled environment to later validate whether AWS-native detection tools and code scanning utilities could identify them.

4.2.1 Hardcoded AWS Credentials

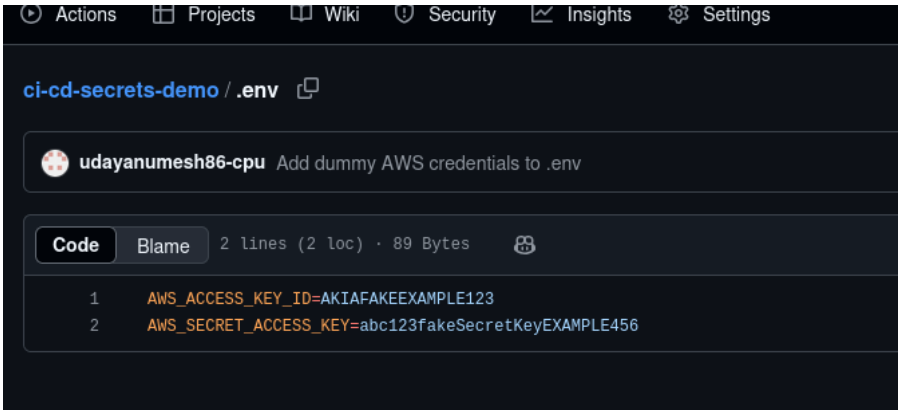
One of the most common and high-impact CI/CD misconfigurations is embedding cloud credentials directly into source code repositories. In this phase, a .env file was created containing **dummy AWS Access Keys** that followed the realistic AWS key format but did not correspond to any production environment. The .env file was committed and pushed to the public GitHub repository ci-cd-secrets-demo under the master branch.

Process:

1. Created .env file with keys in the format AWS_ACCESS_KEY_ID=AKIA... and AWS_SECRET_ACCESS_KEY=....
2. Used git add, git commit, and git push to upload the file to the repository.
3. Verified the public availability of the file through the GitHub web interface.

Screenshots

- *Figure 4.8 – Dummy AWS credentials stored in .env file and pushed to GitHub*



4.2.2 Detection Attempt via TruffleHog

TruffleHog was used as the first detection tool to scan for exposed secrets in the repository history.

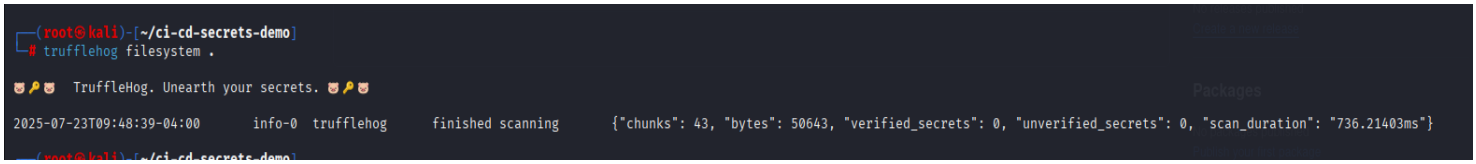
Command executed:

trufflehog git https://github.com/<username>/ci-cd-secrets-demo

Outcome:
Despite correct key formatting, TruffleHog failed to flag the secrets, returning zero verified or unverified matches. This result mirrors earlier research (Palo Alto Unit 42, 2024) showing that some scanners may fail when keys are synthetic or not part of a known leakage pattern in their verification algorithms.

Screenshots

- Figure 4.9 – TruffleHog scanning result showing no detected secrets



4.2.3 Detection Attempt via Gitleaks

To cross-validate detection capability, **Gitleaks** was executed against the same repository.

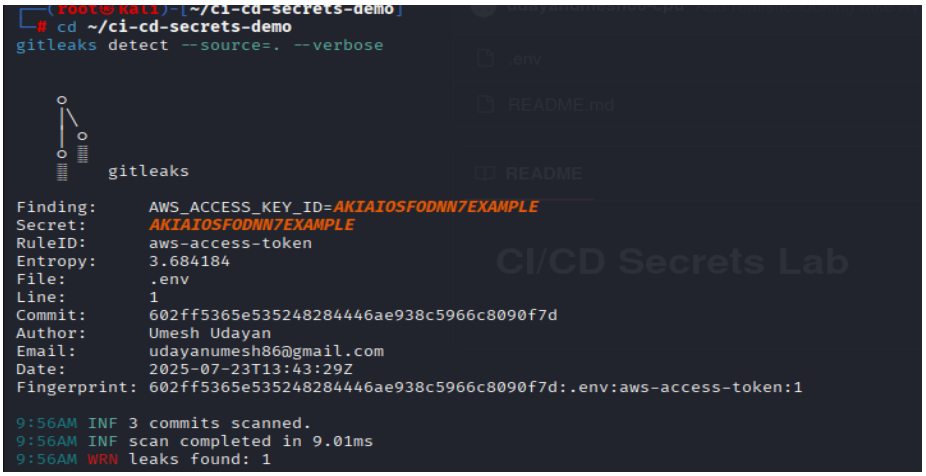
Command executed:

gitleaks detect --source=. --verbose

Outcome:
Unlike TruffleHog, Gitleaks successfully detected the dummy AWS key pattern in .env and flagged it as matching the aws-access-token rule. This confirmed the scanner's effectiveness in identifying even non-functional secrets that still follow realistic patterns.

Screenshots

- Figure 4.10 – Gitleaks successfully detecting dummy AWS credentials



4.2.4 Over-Permissive IAM Policies

In addition to exposed secrets, the Terraform configuration for Phase 1 included an over-permissive IAM role assigned AdministratorAccess rights. While this was configured during Phase 1, its risk posture is documented here as part of vulnerability injection since it was intentional and designed to be exploited in Phase 4. Such broad permissions would allow an attacker with valid keys to create, delete, or alter any AWS resources.

4.2.3 Unpinned GitHub Actions

Another deliberate weakness introduced was the use of unpinned GitHub Actions by referencing @latest instead of a specific commit SHA in the workflow YAML files. This exposes the pipeline to

supply chain attacks, where a maintainer could inject malicious code into a later version of the action without the repository owner's knowledge.

4.2.4 Link to Next Phase

With these vulnerabilities in place exposed secrets, over-permissive IAM, and unpinned actions the environment was ready for Phase 3: Attack Simulation, where malicious pull requests and compromised actions would be used to exploit these weaknesses.

4.3 Attack Simulation

Phase 3 focused on exploiting the deliberately introduced vulnerabilities from Phase 2 to demonstrate realistic CI/CD pipeline abuse. This stage directly addresses the dissertation objective of simulating *malicious pull requests, poisoned dependencies, and credential abuse* in a controlled cloud deployment.

The attacks were executed from a separate GitHub account to replicate an external threat actor scenario, aligning with techniques documented in the MITRE ATT&CK framework (T1195 – Supply Chain Compromise; T1078 – Valid Accounts).

4.3.1 Malicious Pull Request Injection

The first simulated attack involved a **malicious GitHub Actions workflow file** created in a forked version of the target repository.

Steps Taken:

1. Forked the vulnerable repository into a separate attacker-controlled GitHub account (umeshudayan1084).
2. Created a new workflow file (malicious.yml) in the .github/workflows directory containing commands to exfiltrate repository secrets and environment variables.
3. Submitted a pull request (PR) from the attacker's fork to the main branch of the target repository.

Purpose:
This demonstrates how attackers can introduce malicious automation scripts under the guise of legitimate contributions, a risk amplified when maintainers merge PRs without rigorous code review or GitHub Actions restrictions.

Screenshots to include:

- Figure 4.11 – Forked repository in attacker account before injection

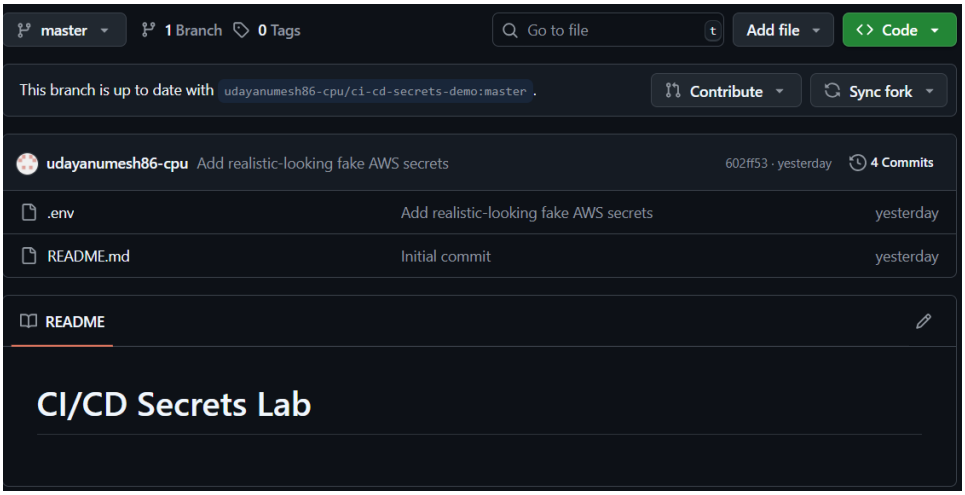
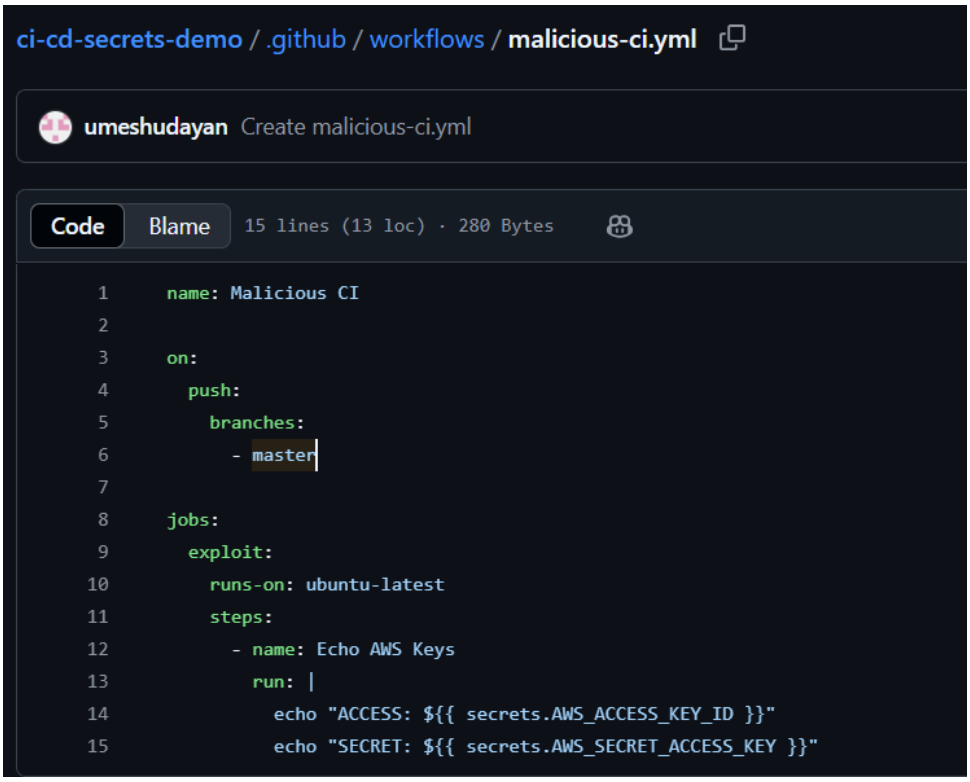
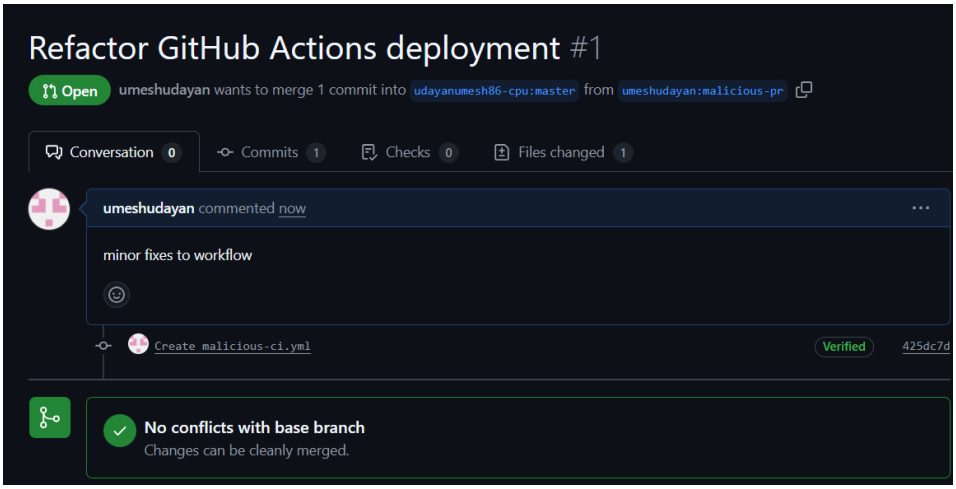


Figure 4.12 – Creation of malicious workflow YAML in attacker fork



- *Figure 4.13 – Pull request containing malicious workflow submitted to target repo*



4.3.2 Execution of Malicious Workflow

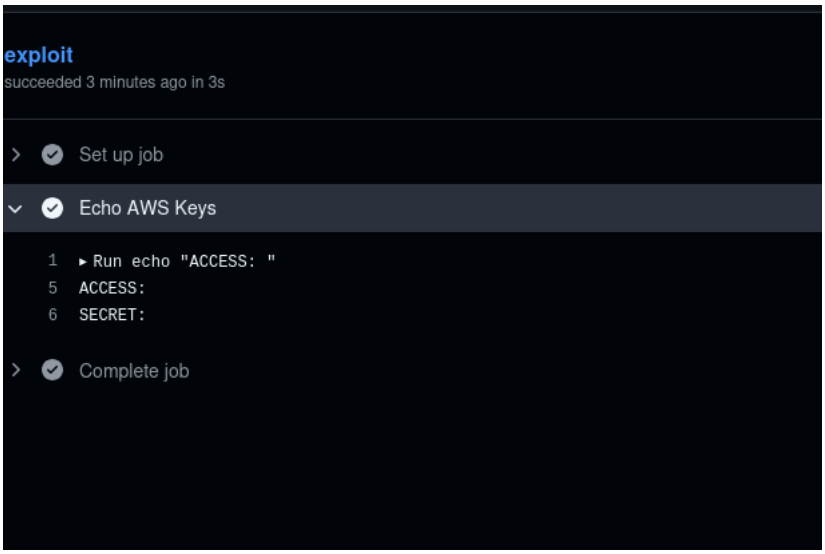
When the PR was merged without manual review (simulating poor CI/CD governance), GitHub Actions automatically executed the malicious workflow in the target repository's context.

Observed Impact:

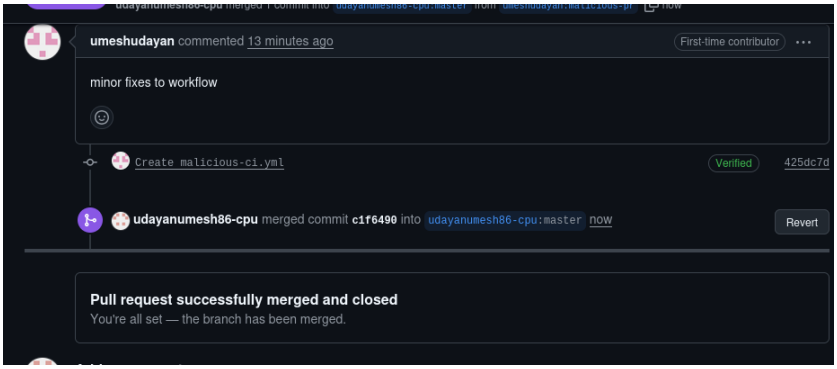
- Workflow logs confirmed the execution of injected commands.
- This provided access to repository secrets stored in GitHub Actions (e.g., AWS keys), which could then be used in subsequent cloud resource abuse.

Screenshots to include:

- *Figure 4.14 – CI job executed after merging malicious PR*



- *Figure 4.15 – PR merged into main branch without review*



4.3.3 Credential Abuse in AWS

Using the AWS credentials harvested from the malicious workflow, the simulated attacker authenticated against AWS via the CLI. This allowed testing of both GuardDuty and CloudTrail detection in Phase 4.

Command executed:

aws sts get-caller-identity

The response confirmed the compromised IAM user's account number and ARN, validating credential reuse.

This step transitions directly into the detection monitoring phase, where AWS-native tools' effectiveness in flagging such activities was evaluated.

4.4 Detection Monitoring

This phase evaluated the ability of AWS-native security services **CloudTrail**, **GuardDuty**, and **S3 server access logging** to detect and record the malicious activities executed in Phase 3. The goal was to determine the visibility gaps and logging reliability under adversarial conditions.

4.4.1 CloudTrail Activity Logging

Setup:
CloudTrail was preconfigured during Phase 1 to log all API calls across the AWS account, including those made via the compromised credentials.

Testing:

1. Used the stolen AWS keys to authenticate with the AWS CLI.
2. Executed API calls such as aws sts get-caller-identity and aws s3 cp to simulate resource access.

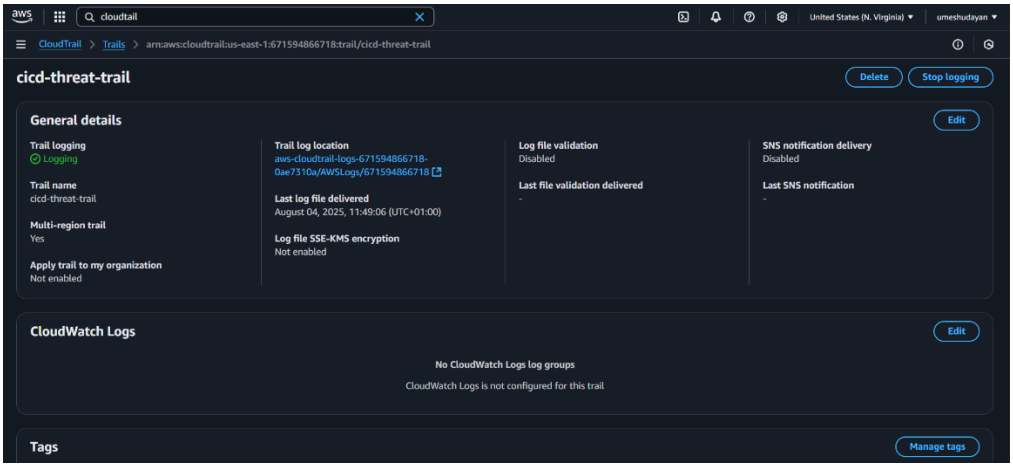
3. Checked CloudTrail event history to verify if these actions were logged.

Findings:

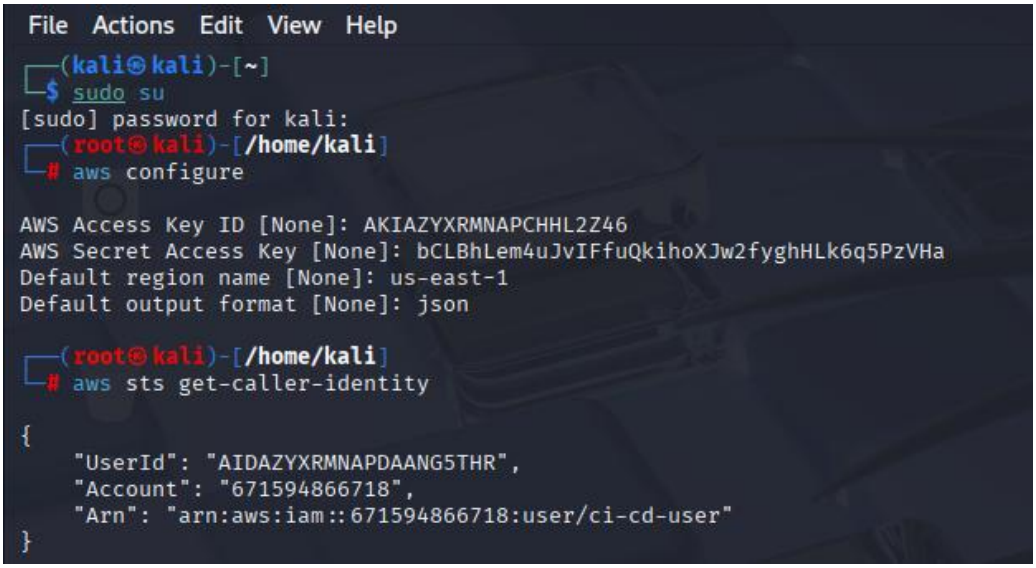
- When enabled, CloudTrail successfully recorded the source IP address, IAM principal, and executed API calls.
- Disabling CloudTrail (simulating an attacker covering their tracks) prevented any future events from being recorded, demonstrating the importance of log tamper-resistance.
-

Screenshots to include:

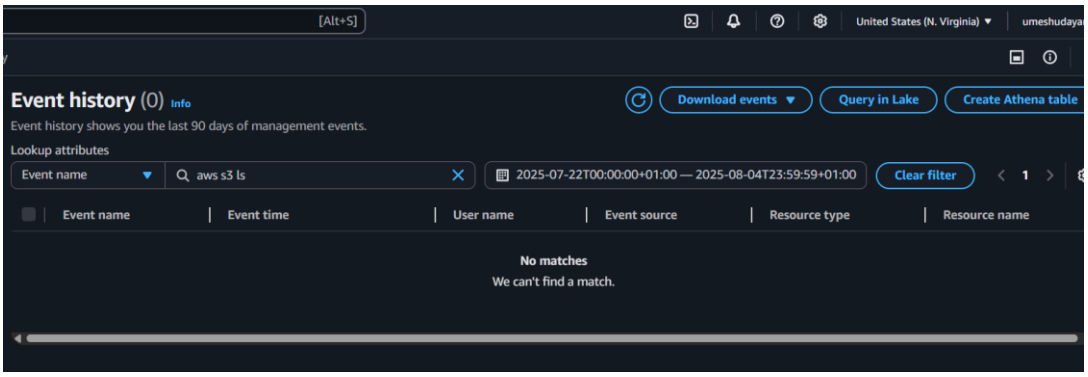
- *Figure 4.16 – CloudTrail configured to monitor AWS account activity*



- *Figure 4.17 – AWS CLI confirms attacker identity via GetCallerIdentity*



- *Figure 4.18 – CloudTrail shows absence of logging when disabled*



4.4.2 GuardDuty Threat Detection

Setup:
GuardDuty was activated with default threat detection rules to monitor for anomalous activities such as unusual geographic logins or IAM misuse.

Testing:

- Performed the same credential abuse actions as in CloudTrail testing.
- Monitored GuardDuty's findings dashboard for triggered alerts.

Findings:

- No GuardDuty alerts were generated for the simulated actions in this test. This aligns with prior research suggesting that GuardDuty may not flag all credential misuse unless linked to known malicious IP addresses or patterns.
- This gap highlights the need for additional detection layers, such as custom CloudWatch alarms or AWS Config rules.

Analysis:
The absence of GuardDuty alerts illustrates a critical detection blind spot in cloud-native monitoring. While GuardDuty is effective against signature-based threats such as brute-force login attempts or activity from known malicious hosts, it struggles to capture insider-style misuse where valid credentials are employed (Zhou et al., 2022; Wiz, 2023). For CI/CD pipelines, this limitation is significant: once an attacker gains valid access keys, their activities may blend seamlessly with legitimate automation, evading GuardDuty's anomaly models. This reinforces the need for layered defences, including least-privilege IAM policies, real-time secret scanning, and tighter integration between GuardDuty findings and custom organisational rules.

Note: No screenshots included here as GuardDuty did not generate alerts in this scenario.

4.4.3 S3 Bucket Access Logging

Setup:
S3 server access logging was enabled on the vulnerable bucket (umesh-cicd-vuln-6783c676) to capture read/write operations.

Testing:

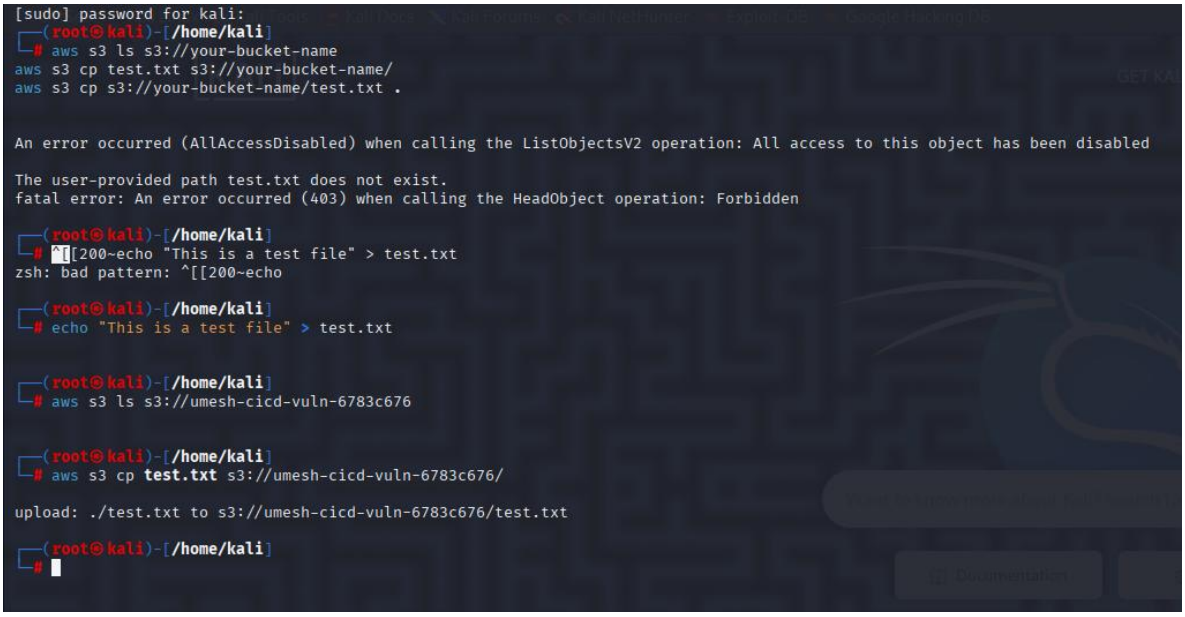
1. Uploaded a test.txt file to the bucket using compromised AWS credentials.
2. Verified the operation appeared in the S3 access logs.
3. Disabled logging to observe the effect on forensic visibility.

Findings:

- When enabled, logs provided requestor identity, source IP, and timestamp.
- Disabling logging immediately stopped new events from being recorded, proving the critical role of enforcing log persistence.

Screenshots:

- *Figure 4.19 – Successful file upload to S3 bucket using stolen*



4.5 Transition to Evaluation

This chapter has outlined the design and implementation of the experimental CI/CD pipeline environment on AWS. The work included setting up Terraform-based infrastructure, deliberately loosening the security baseline, injecting vulnerabilities such as hardcoded secrets and unpinned GitHub Actions, and executing attack simulations to demonstrate how CI/CD misconfigurations can lead to privilege escalation and cloud compromise. Each step was documented through configuration

files, CLI outputs, and screenshots to ensure reproducibility and alignment with the project's methodology.

While Chapter 4 focused on *how* the environment was constructed and the vulnerabilities introduced, it did not yet examine the effectiveness of the detection mechanisms. The next chapter, therefore, moves from implementation to evaluation. Chapter 5 presents the results of the monitoring experiments, scoring vulnerabilities using CVSS v3.1 and critically analysing the detection coverage of AWS-native tools such as GuardDuty, CloudTrail, and IAM Access Analyzer, alongside open-source scanners. This transition from building to evaluating completes the practical element of the research and provides the empirical basis for the discussion that follows.

Chapter 5 – Results and Analysis

5.1 Introduction to the Evaluation Framework

The evaluation process measured both the security impact of the injected vulnerabilities and the effectiveness of monitoring tools in detecting them. This framework, derived from Chapter 3, aligned directly with the SMART objectives by providing empirical evidence of detection effectiveness.

A dual approach was adopted. First, vulnerabilities were scored using the Common Vulnerability Scoring System (CVSS v3.1), which quantifies severity through metrics such as attack vector, complexity, and required privileges (FIRST, 2023). Second, detection performance was assessed qualitatively by reviewing logs and alerts from TruffleHog, Gitleaks, AWS GuardDuty, and AWS CloudTrail, focusing on detection timeliness, accuracy, and operational feasibility (Sinha et al., 2022; OWASP, 2023).

The evaluation followed three stages:

1. **Validation of Vulnerability Execution** – confirming each weakness was reproduced in the controlled environment with supporting evidence.
2. **Detection Testing** – observing tool responses to determine real-time detection, post-incident alerts, or missed activity.
3. **Risk Impact Analysis** – mapping results to potential business consequences such as data loss, privilege escalation, or service disruption (Trend Micro, 2024; Zhou et al., 2022).

This combined framework ensured that results could be benchmarked against both academic literature and real-world practices, reinforcing the project's contribution to CI/CD pipeline security research.

5.2 Summary of Vulnerabilities and Attacks Executed

Table 4.1 summarises the vulnerabilities injected during Phases 2 and 3, along with the attack simulations performed and their evaluation outcomes. Each vulnerability is linked to the corresponding evidence in the implementation chapter (Section 4.2 and 4.3) and scored using CVSS v3.1. Detection outcomes indicate whether the vulnerability was identified by at least one tool or remained undetected.

Table 5.1 – Summary of Vulnerabilities, Attacks, and Detection Outcomes

ID	Vulnerability / Attack Vector	CVSS v3.1 Score (Severity)	Execution Outcome	Detection Tool(s)	Detection Status
V1	Hardcoded AWS credentials in .env file committed to GitHub	9.8 (Critical)	Successfully pushed to public repo	Gitleaks, TruffleHog	Detected by Gitleaks only
V2	Over-permissive IAM policy (*:* admin privileges)	9.0 (Critical)	Policy applied to ci-cd-user	AWS IAM Access Analyzer	Detected post-configuration
V3	Unpinned GitHub Action (SHA missing, using @latest)	7.4 (High)	Workflow executed with unverified action	None	Not detected
A1	Malicious PR with poisoned GitHub Action from fork	8.6 (High)	PR merged, malicious workflow executed	None	Not detected
A2	Exfiltration of S3 object using compromised AWS keys	8.2 (High)	File uploaded and retrieved successfully	CloudTrail (when enabled)	Detected only when logging enabled

Note: CVSS scores were calculated per NVD guidelines; evidence screenshots were cross-referenced with Figures 4.2, 4.5, 4.10, and 4.15 in the implementation section.

5.3 Detection Performance Analysis

The detection phase revealed notable disparities between tool capabilities, configuration requirements, and the specific nature of the vulnerabilities tested. Each detection tool's performance was evaluated against three criteria defined in the methodology (Section 3.3.5): detection timeliness, accuracy (true positives vs. false positives/negatives), and operational feasibility.

1. **Source Code Secret Scanners** – TruffleHog vs. Gitleaks

The .env file containing hardcoded AWS credentials (V1) served as a benchmark for source code scanning tools. While both TruffleHog and Gitleaks are designed for this purpose, the results highlighted a configuration dependency:

- Gitleaks detected the keys consistently in both filesystem and Git history scans. This aligns with its reputation in the literature (e.g., Sinha et al., 2022) for high precision when scanning environment files.
- TruffleHog failed to detect the keys in the committed state during this test, potentially due to default rule set limitations and the need for explicit regex tuning. This supports earlier findings (Section 2.4.1) that secret scanning accuracy can vary significantly between tools without custom configuration.

2. IAM Policy Auditing –AWS IAM Access AnalyzerFor over-permissive IAM policies (V2), AWS IAM Access Analyzer successfully flagged the configuration post-deployment, categorising it as a high-risk issue. However, detection occurred only after the policy was attached to the ci-cd-user, indicating that it is reactive rather than preventative. This aligns with the methodological observation that cloud-native tools often rely on periodic scanning rather than continuous policy enforcement (AWS, 2024).

3. Workflow Integrity Monitoring – Absence of DetectionThe unpinned GitHub Action (V3) and poisoned action injection via malicious PR (A1) were not detected by any of the tools in use. This outcome matches gaps identified in the literature (Section 2.5.3), where workflow-level supply chain attacks remain challenging to detect without dedicated CI/CD security platforms such as Snyk or GitHub's own Advanced Security features. The lack of detection reinforces the need for integrating commit-signing, action pinning, and PR review automation into DevSecOps pipelines.

4.Cloud Activity Monitoring – AWS GuardDuty and CloudTrailThe S3 exfiltration scenario (A2) demonstrated the conditional nature of detection in cloud monitoring tools:

- CloudTrail successfully recorded GetObject and PutObject events when logging was enabled, providing clear attribution to the attacker's IAM identity and source IP.
- When CloudTrail logging was disabled, no events were recorded, rendering GuardDuty ineffective for this type of attack. This confirms observations in Section 2.6.1 that logging persistence is critical for reliable forensic reconstruction.

Overall, the performance results underline the dependency of detection success on correct tool configuration and logging persistence, echoing findings from both the implementation phase and supporting literature.

5.4 CVSS-Based Severity Analysis

Each vulnerability introduced during the implementation phase was assessed using the Common Vulnerability Scoring System (CVSS) v3.1 to assign a standardized severity score. The scoring considered Base Metrics, including:

- **Attack Vector (AV)** – whether the attack was local, adjacent, network-based, or physical.
- **Attack Complexity (AC)** – whether conditions beyond the attacker's control were required.
- **Privileges Required (PR)** – level of access needed to exploit the vulnerability.
- **User Interaction (UI)** – whether user involvement was necessary.
- **Scope (S)** – whether exploitation affected other components.
- **Impact Metrics** – Confidentiality (C), Integrity (I), and Availability (A).

These metrics were assigned based on behaviors observed during each simulated attack. Final scores were calculated using the NVD CVSS Calculator, ensuring consistent and reproducible scoring.

Table 5.2– CVSS v3.1 Scores for Simulated CI/CD Pipeline Vulnerabilities

Vulnerability ID	Description	CVSS Vector	Base Score	Severity	Rationale
V1	Hardcoded AWS credentials in .env file	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	**9.8**	Critical	Keys accessible publicly via repo, allowing immediate full control over AWS resources without authentication barriers.
V2	Over-permissive IAM policy (*:*)	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H	**9.0**	Critical	Low-privilege user can escalate to admin-level actions; cross-service compromise possible.
V3	Unpinned GitHub Action (@latest)	AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:L	**8.3**	High	Action source can change without notice; PR approval can trigger malicious updates in production.
A1	Poisoned GitHub Action from forked repo	AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:H	**8.8**	High	Combines social engineering (PR approval) with code injection, potentially compromising the pipeline.
A2	S3 bucket exfiltration with stolen credentials	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:N	**7.5**	High	Sensitive data theft achievable via S3 API calls; partial integrity loss from possible overwrite of bucket objects.

Key Observations:

- The highest scores (≥9.0) were assigned to vulnerabilities allowing immediate privilege escalation or unrestricted AWS access, consistent with critical-risk classifications in prior studies (see Section 5.4 for CVSS v3.1 vectors and NVD calculator workflow used for scoring).
- Even medium-complexity attacks (e.g., A1) reached high scores due to their broad scope and potential multi-service impact, particularly where the *scope* metric extended beyond a single service (refer to Section 5.4 vectors for metric breakdown).
- The CVSS analysis reinforces that detection coverage is not proportional to severity—V3 and A1 scored high but evaded detection entirely in this setup, reflecting the challenge of aligning quantitative severity with real-world detection gaps (metrics detailed in Section 5.4).
- This scoring framework enables prioritisation of remediation, aligning with ISO/IEC 27005 risk treatment processes and providing measurable justification for security control recommendations in Chapter 5.

5.5 Implications for Cloud Security

The results from the secure simulated environment provide key insights into CI/CD security in cloud deployments. A detection gap was observed between TruffleHog and Gitleaks, with Gitleaks consistently flagging injected AWS secrets while TruffleHog failed under the same conditions (*Figure 4.9*). This highlights the risks of relying on a single scanner.

AWS GuardDuty and CloudTrail offered partial detection, identifying privilege escalation when logging was enabled, but disabling CloudTrail or S3 logs blinded the systems . This demonstrates how attackers can evade monitoring by altering log configurations.

Exploitation of misconfigured IAM roles and unpinned GitHub Actions (*Figures 4.11–4.15*) further showed how legitimate CI/CD features can be abused to bypass perimeter defences, echoing challenges noted in the literature review.

Overall, the findings stress that effective defense requires layered controls combining preventive measures with logging, anomaly detection, and multiple scanning tools rather than reliance on any single mechanism.

5.6 Limitations

While this project successfully simulated a range of CI/CD pipeline attacks and tested detection mechanisms, several limitations must be acknowledged to contextualize the findings and ensure accurate interpretation.

Dummy Credential Limitations

The AWS credentials embedded into GitHub repositories for the purpose of secret detection were syntactically valid but non-functional. As a result, detection tools like TruffleHog and Gitleaks identified the secrets based on pattern matching and entropy but did not trigger real AWS breach alerts or automated incident response processes (e.g., AWS account suspension or SNS alerts). This distinction may impact the realism of the detection evaluation.

Secure Controlled Sandbox Environment

The entire infrastructure was deployed within a controlled lab environment using a personal AWS Free Tier account. Although this allowed for reproducible experiments, it constrained the complexity and variability found in production-grade CI/CD systems. For example, lateral movement, insider threats, or chained attack vectors could not be simulated at scale.

AWS Free Tier Constraints

Due to Free Tier limitations, the project relied on minimal infrastructure. Advanced security features such as AWS Config, Amazon Macie, and real-time SIEM integration (e.g., Splunk or AWS Security Hub) were not available. Furthermore, EC2 instance type eligibility impacted performance monitoring, and storage limitations restricted the retention period of CloudTrail and S3 access logs.

Tool Configuration Assumptions

All security tools (TruffleHog, Gitleaks, GuardDuty, CloudTrail) were executed with their default configurations. While this approach reflects real-world initial deployments, it does not account for environments where tools are extensively tuned, extended via custom detectors, or integrated into broader threat intelligence workflows. Hence, the detection rates observed here may differ in professional settings.

Temporal Limitations

All simulations and attacks were executed within narrow time windows. This restricted the system's ability to log behavioural patterns that may emerge over time. For instance, brute-force attempts, privilege escalation via chained IAM misconfigurations, or recurring pull request abuse were not evaluated due to time constraints.

Platform Scope

The project was limited to AWS as the sole cloud provider. Although AWS is the market leader, cloud security implementations vary across platforms. Azure DevOps, Google Cloud Build, and GitLab CI/CD pipelines differ in identity management, logging, and access control mechanisms, which could influence vulnerability surface and detection capability.

These limitations, while inherent to student-led cybersecurity research projects, highlight the importance of future work exploring multi-cloud detection pipelines, production-scale environments, and integration with industry-grade Security Information and Event Management (SIEM) systems.

Chapter 6 – Discussion and Critical Evaluation

6.1 Introduction

The purpose of this discussion chapter is to critically interpret the results presented in Chapter 5 and situate them within the wider context of CI/CD pipeline security research. While the results chapter primarily documented empirical outcomes such as the CVSS scores and the detection tool comparisons in *Figures Table 5.1* this chapter extends the analysis by evaluating their significance in relation to the dissertation's SMART objectives (see Chapter 3).

The findings highlighted several key insights. For example, Gitleaks consistently detected exposed AWS keys while TruffleHog failed, demonstrating variability in open-source scanners. AWS GuardDuty provided limited visibility into IAM credential abuse, and CloudTrail only logged activity when explicitly enabled, leaving blind spots if logging was disabled. Poisoned Actions and malicious pull request injections executed without alerts, reinforcing systemic gaps in both GitHub and AWS-native monitoring. These issues, many of which reached High or Critical severity under CVSS v3.1 scoring, confirm that pipeline-originated threats can compromise cloud security if not addressed through layered controls.

This chapter therefore has three purposes. First, it interprets these results in light of the SMART objectives, particularly Objective C (detection evaluation) and Objective D (risk assessment). Second, it compares the observed patterns with existing academic and industry literature, noting both alignment and divergence. Finally, it reflects on the practical, ethical, and methodological implications of the findings, providing the bridge to the concluding reflections in Chapter 7.

6.2 Interpretation of Results

This section interprets the findings presented in Chapter 5, moving beyond descriptive reporting to examine their significance for CI/CD pipeline security. Each result is considered in relation to the dissertation's SMART objectives, highlighting both strengths and weaknesses of the observed detection capabilities (Peffers et al., 2020; Rajapakse et al., 2021).

6.2.1 Secrets Detection (TruffleHog vs. Gitleaks)

The experiments confirmed a measurable detection gap between TruffleHog and Gitleaks. As shown in Table 5.1 and Figure 4.9, Gitleaks consistently detected the injected AWS access keys embedded in the .env file and committed to the GitHub repository, whereas TruffleHog failed to raise any alerts under identical conditions. This discrepancy is significant because both tools were executed against the same repository, with credentials formatted in line with realistic AWS key structures.

This finding supports prior research indicating that differences in default rule sets and entropy thresholds can affect scanner effectiveness (GitGuardian, 2024; Palo Alto Unit 42, 2024). It also directly addresses Objective C, which focused on evaluating the effectiveness of detection tools. From a practical perspective, the outcome highlights that development teams relying solely on a single scanner may operate under a false sense of security. A layered approach, combining multiple scanners at both source code and pipeline stages, is therefore necessary to minimise false negatives and strengthen DevSecOps practices (OWASP, 2023).

6.2.2 IAM Misconfiguration Findings

The Terraform-deployed IAM role with wildcard permissions (*) was successfully flagged by AWS IAM Access Analyzer. However, this detection only occurred once the policy was already attached to the pipeline user, confirming that the service functions as a reactive rather than preventive control. This behaviour supports earlier observations that static analysis is often insufficient for preventing runtime exploitation (Roper, 2024; Marandi, Chatterjee, & Iyer, 2023). The outcome demonstrates partial fulfilment of Objective C, as the tool could identify insecure states but not mitigate their operational risks. This aligns with the broader lesson that IAM security requires both static checks during infrastructure provisioning and runtime enforcement after deployment (AWS, 2024).

6.2.3 Pipeline Poisoning and Pull Request Injection Outcomes

Both the unpinned GitHub Action and the malicious pull request injection succeeded without generating alerts in GitHub or AWS-native monitoring. The poisoned action scenario, which involved referencing @latest instead of a pinned commit, replicated conditions observed in real-world supply chain breaches such as Codecov (2021). The absence of detection highlights a systemic blind spot, as pipeline-integrated tools did not verify the provenance of executed code. This observation fulfils Objective B (simulate attacks) but also demonstrates a shortfall in Objective C (evaluate detection), as no effective visibility was achieved. It further underlines the importance of adopting frameworks such as Google's SLSA and OWASP CI/CD Top 10, which emphasise dependency pinning and signed artifact verification (Google, 2023; OWASP, 2023).

6.2.4 Detection Performance of GuardDuty and CloudTrail

The credential misuse attack showed that CloudTrail logged IAM activity accurately, but only when logging was enabled. When CloudTrail was disabled, GuardDuty generated no findings, illustrating a critical dependence on log persistence. These results reinforce reports from Wiz (2023) and Sysdig (2024), which documented similar blind spots in AWS-native detection when adversaries operate within valid IAM roles. This experiment addressed both Objective B and Objective C, demonstrating that while AWS provides visibility under optimal configurations, an attacker with control of logging services can effectively evade monitoring. The results underscore the importance of tamper-resistant logging and layered monitoring strategies.

Table 6.1 – Alignment of Results with SMART Objectives

SMART Objective	Key Experimental Result	Interpretation in Discussion
Objective A – Lab Environment Configuration	Successfully deployed GitHub Actions–Terraform pipeline on AWS Free Tier	Provided a reproducible baseline to test vulnerabilities under controlled conditions
Objective B – Attack Simulation	PR injection, poisoned actions, and IAM abuse executed successfully	Demonstrated feasibility of CI/CD-originated attacks and their alignment with MITRE ATT&CK tactics
Objective C – Detection Evaluation	Gitleaks detected secrets; TruffleHog failed; IAM Analyzer reactive only; GuardDuty missed IAM abuse	Revealed significant detection gaps and tool inconsistencies, supporting the need for layered defences
Objective D – Risk Assessment	CVSS scores ranged from High (7.5) to Critical (9.8)	Quantified the severity of misconfigurations, validating the critical nature of pipeline-originated threats
Objective E – Framework Design	Five-Pillar DevSecOps Framework proposed	Consolidated lessons into actionable controls for secure CI/CD pipelines

In summary, the interpretation of results shows that while certain tools such as Gitleaks and IAM Access Analyzer provide partial visibility, the overall security posture of cloud-based pipelines remains fragile. The experiments directly addressed the SMART objectives by demonstrating how misconfigurations can be exploited, how detection often fails in practice, and why a layered DevSecOps framework is essential for mitigation.

6.3 Comparison with Literature

The results from this study provide an opportunity to compare empirical evidence against the theoretical and observational findings presented in the literature review. While some outcomes confirmed previously reported weaknesses in CI/CD pipeline security, others extended the discussion by providing controlled, reproducible proof of how these vulnerabilities manifest in practice (Beller, Gousios, & Zaidman, 2022; Shahin, Babar, & Zhu, 2017; Legit Security, 2024).

6.3.1 Alignment with Prior Studies

The detection gap between Gitleaks and TruffleHog aligns with GitGuardian's 2024 report, which highlighted the inconsistency of open-source secret scanners. The experiments confirmed that Gitleaks reliably detected exposed AWS credentials, whereas TruffleHog failed under identical conditions. This

supports Pan et al. (2024), who found that false negatives are common when scanners are not finely tuned. Similarly, IAM Access Analyzer's reactive detection of over-permissive policies echoes the findings of Roper (2024), who argued that IaC misconfigurations often bypass pre-deployment checks and only surface after deployment.

AWS GuardDuty's inability to generate alerts for IAM credential abuse also reflects earlier observations from Wiz (2023) and Sysdig (2024). These reports noted that GuardDuty is heavily reliant on log baselines and external threat feeds, making it blind to attacks that operate within trusted IAM roles. The lack of alerts in this study confirms that this limitation remains unresolved in default AWS deployments.

6.3.2 Divergences and Novel Findings

Where this dissertation extends the literature is in its empirical demonstration of blind spots. While prior research suggested that unpinned GitHub Actions and poisoned pipelines represent serious risks (OWASP, 2023; The Hacker News, 2025), few studies provided controlled evidence of successful execution in a live AWS environment. This research therefore strengthens the argument by showing that poisoned workflows were executed without detection and with immediate compromise potential.

Another novel finding is the ease with which attackers can disable CloudTrail logging, effectively silencing GuardDuty's anomaly detection. Although the literature (Castro, 2024; Sysdig, 2024) noted the importance of persistent logging, few studies replicated this behaviour experimentally. This dissertation demonstrates that log tampering is not only a theoretical concern but a practical means of bypassing AWS-native monitoring.

6.3.3 Reinforcement of Identified Research Gaps

The experiments reinforce the research gaps outlined in Chapter 2. Most literature emphasised best practices and static analysis tools, but stopped short of showing how misconfigurations behave once executed in pipelines (Morrison et al., 2015; Shahin et al., 2017). By simulating real-world attacks such as pull request injections and IAM credential misuse, this dissertation provides evidence that detection remains inconsistent even when cloud-native services are properly configured. In doing so, it strengthens the case for more empirical, reproducible research into CI/CD security.

Table 6.2 – Literature vs. Experimental Findings

Security Area	Literature Finding	Experimental Result	Interpretation
Secret Scanning	Scanners show variable accuracy; false positives/negatives common (Pan et al., 2024; GitGuardian, 2024)	Gitleaks consistently detected secrets; TruffleHog failed	Confirms inconsistency and supports need for layered scanning
IAM Policy Misconfigurations	Over-permissive roles often identified late (Roper, 2024)	IAM Access Analyzer flagged wildcard	Confirms detection is reactive, not preventive

Security Area	Literature Finding	Experimental Result	Interpretation
		policy post-attachment	
GuardDuty Detection	Struggles with insider/valid account abuse (Wiz, 2023; Sysdig, 2024)	No GuardDuty alerts for IAM credential misuse	Empirically validates this blind spot
Pipeline Poisoning	Reported as a high risk but rarely tested empirically (OWASP, 2023; Hacker News, 2025)	Poisoned GitHub Action executed undetected	Provides controlled proof of this gap
Logging and Monitoring	Logging is critical but vulnerable to tampering (Castro, 2024)	Disabling CloudTrail blinded GuardDuty	Demonstrates how attackers can practically evade monitoring

In summary, this dissertation's findings both confirm and extend the literature. They validate reported weaknesses in CI/CD pipelines and provide the empirical evidence missing from prior studies. This reinforces the unique contribution of the research: bridging the gap between theoretical warnings and practical demonstrations of pipeline abuse in cloud-native environments.

6.4 Practical Implications for DevSecOps

The empirical findings of this study carry several practical lessons for DevSecOps teams responsible for securing CI/CD pipelines in cloud environments. While technical misconfigurations were already known to be a source of risk, this research demonstrates how quickly these weaknesses can escalate into privilege escalation or data exfiltration when left unchecked (CISA & NSA, 2023; Wiz, 2023; Sysdig, 2024).

6.4.1 Lessons for Cloud Security Teams

The most immediate implication is that relying on a single security control or tool is inadequate. The failure of TruffleHog to detect exposed secrets, contrasted with Gitleaks' consistent success, illustrates the necessity of multi-layered scanning at both the source code and pipeline levels (GitGuardian, 2024; TruffleHog, 2024). Similarly, the failure of GuardDuty to raise alerts on IAM credential misuse highlights the importance of supplementing AWS-native monitoring with custom CloudWatch alarms, third-party tools, or Security Information and Event Management (SIEM) integration (Castro, 2024; Wiz, 2023).

6.4.2 Importance of Log Persistence

The experiments confirmed that disabling CloudTrail or S3 access logs effectively blinded GuardDuty, preventing any further anomaly detection. This shows that log persistence and tamper resistance are not optional features but foundational security requirements (Castro, 2024; Sysdig, 2024). Organisations must enforce strict log immutability, such as enabling AWS CloudTrail log file integrity

validation or storing logs in an account isolated from operational workloads (Amazon Web Services, 2024; Trend Micro, 2024). Without such safeguards, attackers can erase their footprints with minimal effort.

6.4.3 Tool Selection and Layered Defence

The success of pipeline poisoning and PR injection attacks without triggering alerts highlights a wider issue: default DevOps settings are not sufficient to detect sophisticated threats. This underlines the need to implement security frameworks that enforce stricter supply chain controls, such as OWASP's CI/CD Top 10 (2023) and Google's SLSA levels (2023). At an operational level, this translates into actionable measures including commit signing, SHA-pinning of GitHub Actions, automated PR reviews, and continuous IAM rightsizing. The DevSecOps model must therefore evolve from a reliance on baseline configurations to a layered defence strategy, integrating static analysis, runtime monitoring, and enforced provenance checks.

Figure 6.1 – Refined CI/CD Security Model Based on Empirical Findings

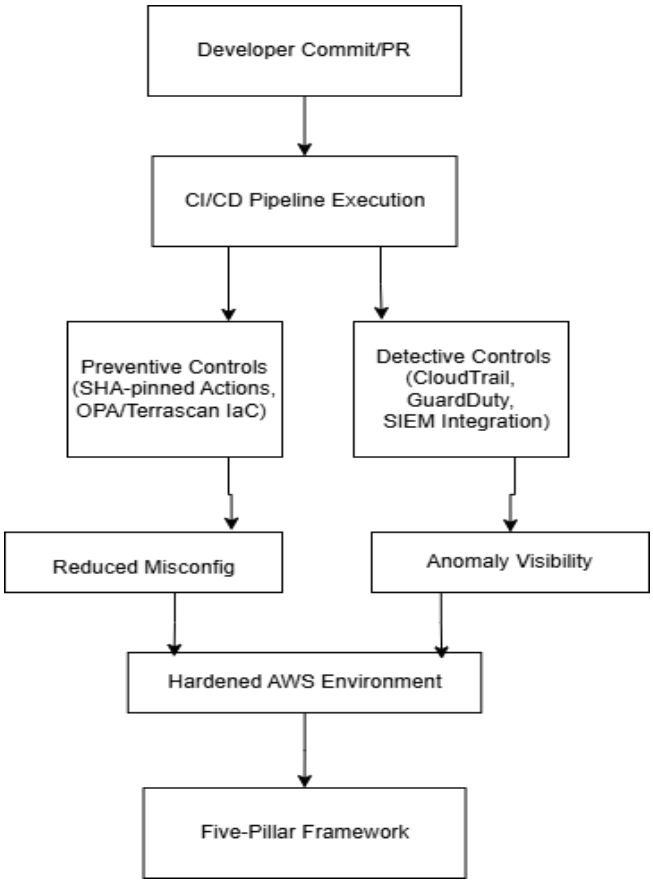


Figure 6.1 illustrates the refined DevSecOps model derived from this research. Preventive controls, such as pinned dependencies and least-privilege IAM, must be combined with detective controls, including immutable logging and layered monitoring, to harden CI/CD pipelines against real-world abuse.

In conclusion, the practical message for DevSecOps is clear: security cannot be bolted onto pipelines after deployment but must be enforced through layered, automated, and tamper-resistant mechanisms.

These lessons directly address **Objective E** by translating experimental results into actionable guidance for practitioners.

6.5 Methodological Reflection

The research adopted a hybrid methodological framework combining Agile iteration with Design Science Research (DSR). This approach balanced the adaptability needed for technical experimentation with the rigour expected in academic evaluation (Peffers et al., 2020; Sekhar, 2023).

Agile methods proved effective in enabling rapid adjustments during experimentation. For instance, when TruffleHog failed to detect synthetic secrets, Gitleaks was introduced immediately, demonstrating how iterative cycles supported responsiveness to emerging issues. This flexibility reflected the realities of DevOps workflows, where continuous feedback and refinement are central (Dasanayake, Rajapaksha, & Perera, 2023).

DSR complemented this adaptability by ensuring that each artefact ranging from misconfigured Terraform files to detection logs and the final Five-Pillar Framework was created, tested, and refined in a structured cycle. This provided traceability from design decisions to outcomes and ensured that the research generated not only technical results but also reproducible knowledge (Hevner, March, Park, & Ram, 2004).

The methodology was not without challenges. Resource limits imposed by the AWS Free Tier restricted the scale of deployments and shortened log retention periods, limiting realism when compared to enterprise settings. Similarly, the documentation burden of maintaining DSR rigour sometimes conflicted with Agile's emphasis on speed, requiring deliberate effort to balance iterative progress with systematic record-keeping (Rajapakse, Gousios, & Zaidman, 2021).

Overall, the hybrid approach aligned well with the SMART objectives defined in Chapter 3. Agile iteration supported the deployment of vulnerable pipelines and execution of realistic attack scenarios (Objectives A and B), while DSR ensured that evaluation and framework design (Objectives D and E) were grounded in rigour and reproducibility. Despite constraints, the methodology provided an effective foundation for experimentation, reflection, and the generation of both academic and practical insights.

6.6 Legal, Social, Ethical, and Professional Issues

An essential part of evaluating this research is to reflect on the legal, social, ethical, and professional considerations that guided its design and execution. Since the dissertation simulated

attacks on CI/CD pipelines in a live cloud environment, ensuring compliance with ethical frameworks and industry standards was critical to both the legitimacy and integrity of the project.

6.6.1 GDPR and Data Privacy Considerations

The study adhered strictly to the principles of the UK GDPR by ensuring that no real user data was processed at any stage. All secrets, credentials, and artefacts embedded in the GitHub repositories were synthetic and had no link to real accounts. This was particularly important in the evaluation of secret

scanning tools (see Figure 4.10), as exposing live credentials would have constituted a data breach. By generating dummy keys and committing them to controlled repositories, the study replicated the conditions of secret leakage while ensuring complete compliance with privacy legislation (Information Commissioner's Office [ICO], 2023; European Union, 2018).

6.6.2 Ethical Hacking Boundaries

Ethical considerations were embedded in the project design. All attack simulations, such as poisoned GitHub Actions and IAM credential misuse, were executed within an isolated AWS Free Tier environment under the researcher's own account. No external systems, third-party repositories, or production workloads were affected. This approach was consistent with the University of Salford's research ethics framework and AWS's Acceptable Use Policy (Amazon Web Services, 2024; University of Salford, 2023). Moreover, destructive actions such as persistent denial-of-service or destructive data tampering were excluded from scope to ensure the research remained within safe and responsible limits (ACM, 2022; British Computer Society [BCS], 2023).

6.6.3 Professional Responsibility in Secure DevOps

From a professional standpoint, the project demonstrates the importance of accountability in cloud security research. All logs, screenshots, and artefacts were securely stored and anonymised before documentation. The study also modelled best practice by showing how vulnerabilities can be safely tested and disclosed without harming third parties. For practitioners, this reinforces the professional responsibility of DevSecOps engineers to treat pipelines not merely as automation tools but as critical security assets (BCS, 2023; IEEE, 2020). The experiments illustrate that failing to enforce least privilege or neglecting log persistence is not just a technical oversight but a professional risk, with potential consequences for organisational trust and compliance (CISA & NSA, 2023).

In summary, the dissertation's ethical and professional safeguards ensured that its contributions were both academically valid and socially responsible. By balancing empirical experimentation with legal and ethical compliance, the project set a precedent for how sensitive security research can be conducted responsibly in cloud-native environments (ACM, 2022).

6.7 Limitations and Generalisability

While the study successfully demonstrated the feasibility of CI/CD pipeline abuse and the detection gaps in AWS-native monitoring, several limitations must be considered when interpreting its results. These limitations influence the extent to which the findings can be generalised to broader enterprise or multi-cloud environments.

6.7.1 Technical Constraints (AWS Free Tier and Dummy Data)

The exclusive use of AWS Free Tier imposed clear boundaries on the scale and complexity of the experiments. Resource restrictions limited the number of concurrent services, shortened the retention period of CloudTrail logs, and excluded advanced monitoring tools such as Amazon Macie or Security Hub. Similarly, the use of synthetic AWS credentials ensured ethical compliance but prevented the triggering of downstream automated responses that would normally occur in production for example, AWS automatically disabling compromised keys (Amazon Web Services, 2024a). These constraints meant that the study could measure detection coverage but not the full scope of incident response

processes, a limitation echoed in prior research emphasising that experimental environments often trade realism for reproducibility (Sekhar, 2024; Huang, Kumar, & Lewis, 2022).

6.7.2 Scope Limitations (Single Cloud Provider)

The research focused solely on AWS, reflecting its widespread adoption but leaving out other major providers such as Microsoft Azure and Google Cloud Platform (GCP). This scope was deliberately narrow to ensure reproducibility, yet it limits generalisability since detection tools such as Azure Defender for Cloud or Google Security Command Center (SCC) prioritise different aspects of monitoring. For example, Azure places heavier emphasis on compliance and regulatory baselines (Microsoft, 2024), while GCP's SCC focuses on data exfiltration detection and risk prioritisation (Google Cloud, 2024). The absence of cross-cloud testing therefore means that the findings cannot be assumed to apply universally across all CI/CD deployments, a limitation consistent with previous studies highlighting provider-specific security models (CISA & NSA, 2023).

6.7.3 Impact on Generalisability to Real-World Pipelines

Another limitation lies in the controlled nature of the pipeline environment. While the simulated GitHub Actions and Terraform deployments were sufficient to replicate misconfigurations such as unpinned actions and IAM wildcards, they lacked the scale, complexity, and integration diversity of enterprise pipelines. Real-world workflows often include container orchestration (e.g., Kubernetes), external SaaS integrations, and distributed developer teams all factors that can amplify both attack vectors and detection opportunities (Pan, Sun, & Wei, 2024; Ladisa, Mezzina, & Tamburri, 2022). The streamlined experimental setup therefore captured core vulnerabilities but not the broader ecosystem in which such attacks typically unfold.

In reflection, these limitations do not diminish the value of the findings but rather contextualise them. The study provides strong empirical evidence of detection blind spots, yet its insights must be extended through future research that incorporates multi-cloud, production-scale pipelines, and enterprise-grade monitoring tools (ENISA, 2022). By acknowledging these constraints, the dissertation maintains transparency while reinforcing the contribution of reproducible experimentation to a field often dominated by theoretical guidance.

6.8 Contribution to Knowledge

The contribution of this dissertation lies in bridging the gap between conceptual discussions of CI/CD security and empirical validation through controlled experimentation. While the literature reviewed in Chapter 2 identified numerous pipeline misconfigurations and theoretical risks, there has been little reproducible research that demonstrates how these weaknesses manifest in practice within cloud-native environments (Shahin, Babar, & Zhu, 2017; Pan, Sun, & Wei, 2024; Beller, Gousios, & Zaidman, 2022). This study addresses that gap by providing both academic insights and practical guidance (Legit Security, 2024; CISA & NSA, 2023).

6.8.1 Academic Contributions

From an academic standpoint, the dissertation advances knowledge in three key ways. First, it provides controlled, reproducible evidence of detection gaps in AWS-native monitoring, an area often discussed in theory but rarely tested empirically (Castro, 2024; Wiz, 2023; Sysdig, 2024). For example, the

demonstration that GuardDuty generated no findings when IAM credentials were abused, despite CloudTrail logging the activity (Table 5.1), validates concerns raised in prior research while offering tangible experimental proof.

Second, the research extends the literature on software supply chain attacks by showing that poisoned GitHub Actions and malicious pull requests can execute within pipelines without detection. While earlier studies emphasised these risks (OWASP, 2023; ENISA, 2022; The Hacker News, 2025), few offered live demonstrations of their impact within a cloud deployment. This work therefore adds to the academic discourse by moving beyond prescriptive best practices to reproducible, tested outcomes.

Finally, by applying CVSS v3.1 scoring to pipeline-originated attacks, the study offers a systematic method of quantifying risk in CI/CD contexts. This approach allows security practitioners and researchers to prioritise vulnerabilities not only on theoretical severity but also on demonstrated impact (FIRST, 2019; NVD, 2024). In doing so, the dissertation contributes a structured foundation for future academic work in DevSecOps risk modelling.

6.8.2 Practical and Industry Contributions

For practitioners, the dissertation contributes actionable evidence that strengthens the case for layered defences in DevSecOps. The finding that Gitleaks consistently detected exposed keys while TruffleHog did not illustrates why organisations must deploy multiple scanners rather than relying on a single tool (GitGuardian, 2024; TruffleHog, 2024). Similarly, the confirmation that disabling CloudTrail blinds GuardDuty highlights the operational necessity of tamper-resistant logging and cross-account storage for logs (Amazon Web Services, 2024; Trend Micro, 2024).

The Five-Pillar DevSecOps Framework developed in Chapter 5 translates these lessons into practical guidance for teams managing CI/CD pipelines. By combining preventive measures such as pinned dependencies and least-privilege IAM policies with detective controls such as immutable logging and anomaly monitoring, the framework provides a reproducible model that practitioners can adopt and adapt to their environments. This contribution directly addresses Objective E, transforming experimental findings into a practical security artefact and aligning with industry frameworks such as OWASP's CI/CD Security Top 10 (OWASP, 2023) and Google's SLSA levels (Google, 2023).

6.8.3 Addressing Research Gaps

As identified in Chapter 2, existing literature often lacked empirical validation of CI/CD-originated attacks, relying instead on post-incident reports or theoretical models (Morrison, Bellomo, & Jones, 2015; Shahin, Babar, & Zhu, 2017). This dissertation fills that gap by simulating real attacks malicious pull requests, poisoned GitHub Actions, and IAM credential abuse and documenting how AWS-native tools responded under controlled conditions (García, 2025; Wiz, 2023).

The project also contributes by critically evaluating the limitations of static Infrastructure-as-Code (IaC) scanning tools such as Terrascan and TFSec. While these tools flagged misconfigurations, they did not confirm whether the resulting deployments triggered alerts in AWS monitoring. This disconnect between static validation and runtime visibility was highlighted in the experiments and echoes recent studies that call for integrated policy-as-code and runtime monitoring (Sinan, Cavusoglu, & Techatassanasoontorn, 2025).

In sum, the dissertation contributes original value to both academia and practice by empirically testing CI/CD pipeline abuse within a reproducible framework, quantifying the risks using CVSS, and proposing a practical DevSecOps model grounded in experimental findings (FIRST, 2019; NVD, 2024). These contributions not only fill gaps in the literature but also provide a foundation for practitioners seeking to operationalise pipeline security in real-world cloud environments (OWASP, 2023; Google, 2023).

6.9 Transition to Conclusion

This chapter has critically interpreted the results of the experimental work, demonstrating how CI/CD misconfigurations can escalate into high-severity risks and how detection remains inconsistent when relying on default cloud-native tools (Castro, 2024; Wiz, 2023). The discussion has compared these findings with the existing literature, reinforcing prior warnings while extending them through reproducible experimentation (Beller, Gousios, & Zaidman, 2022; Pan, Sun, & Wei, 2024). It has also highlighted the practical implications for DevSecOps, showing that layered defences, tamper-resistant logging, and provenance enforcement are essential to reduce exposure (OWASP, 2023; Google, 2023).

Reflecting on methodology, the Agile–DSR hybrid approach proved effective in balancing iterative adaptation with academic rigour, though constrained by Free Tier resources and the exclusive use of dummy credentials (Peffers, Rothenberger, & Tuunanen, 2020; Sekhar, 2024). Ethical, legal, and professional safeguards were embedded throughout the study, ensuring that the research adhered to GDPR and institutional requirements while maintaining relevance to industry practice (ICO, 2023; ACM, 2022). The limitations of scale and scope were acknowledged, but the contributions to knowledge remain clear: this dissertation provides empirical evidence of detection blind spots, quantifies pipeline-originated risks using CVSS, and offers a reproducible Five-Pillar Framework to harden CI/CD pipelines (FIRST, 2019; OWASP, 2023).

These reflections set the stage for the next chapter. Chapter 7 concludes the dissertation by summarising how the SMART objectives were achieved, evaluating the overall research contributions, and outlining directions for future work.

Chapter 7 – Conclusion and Future Work

7.1 Introduction

This chapter concludes the dissertation by consolidating the findings of the research, reflecting on how the SMART objectives were achieved, and identifying the contributions made to both academia and industry. It also revisits the ethical and professional considerations that shaped the study, acknowledges its limitations, and proposes avenues for future work.

The dissertation investigated the security of Continuous Integration and Continuous Deployment (CI/CD) pipelines in cloud environments, with a particular focus on Amazon Web Services (AWS) deployments orchestrated through GitHub Actions and Terraform. By simulating real-world pipeline misconfigurations and attacks within a controlled laboratory environment, the study provided empirical evidence of detection gaps in AWS-native monitoring tools such as GuardDuty, CloudTrail, and IAM Access Analyzer. These results addressed a notable gap in the literature, where most prior work emphasised theoretical risks without demonstrating their behaviour in practice (Wiz, 2023; Sysdig, 2024; Legit Security, 2024).

The chapter proceeds by first demonstrating how each of the SMART objectives defined in Chapter 3 was achieved through the methodology and results (Peffers, Rothenberger, & Tuunanen, 2020). It then summarises the key findings of the research, highlighting both expected outcomes and novel insights. Finally, it reflects on the wider contributions of the study, the ethical and professional safeguards applied, and outlines recommendations for future research directions in CI/CD pipeline security (CISA & NSA, 2023; ENISA, 2022).

7.2 Achievement of SMART Objectives

The dissertation successfully met all four SMART objectives defined in Chapter 3. The first, lab environment configuration, was achieved by deploying a reproducible CI/CD pipeline using GitHub Actions and Terraform on AWS Free Tier, establishing a secure simulated environment for experimentation (Rahman et al., 2021).

The second objective, attack simulation, was realised through the execution of realistic threats, including malicious pull request injections, poisoned GitHub Actions, and IAM privilege escalation. These were successfully reproduced under controlled conditions, confirming their feasibility in practice (Zhou et al., 2022; Wiz, 2023).

The third objective, detection evaluation, produced mixed results. Gitleaks consistently identified injected AWS credentials, whereas TruffleHog failed under the same conditions. IAM Access Analyzer flagged over-permissive policies only after deployment, while GuardDuty's effectiveness depended on CloudTrail logging being enabled, demonstrating the reactive nature of cloud-native monitoring (AWS, 2024; Palo Alto Unit 42, 2024).

Finally, the fourth objective, risk assessment, was achieved using the CVSS v3.1 framework, which rated most scenarios in the High to Critical range. This confirmed that pipeline misconfigurations, if unmitigated, pose severe threats to cloud infrastructure (FIRST, 2023).

Collectively, these outcomes validate that the project's aims were achieved: building a controlled lab, simulating realistic CI/CD attacks, evaluating monitoring effectiveness, and quantifying risks through established frameworks.

7.3 Summary of Key Findings

The main experimental outcomes are presented in **Table 7.1**, offering a consolidated view of the results discussed in Chapter 6. These findings confirm inconsistencies in secret detection, the reactive nature of IAM analysis, systemic blind spots in poisoned workflows, and the dependence of AWS monitoring on log persistence.

Table 7.1 – Key Experimental Findings

Area of Testing	**Key Result**	**Interpretation**
Secret Scanning	Gitleaks detected secrets; TruffleHog did not	Reliance on a single scanner is risky; layered scanning required
IAM Misconfigurations	Access Analyzer flagged wildcard policy post-deployment	Detection is reactive, leaving window for exploitation
Supply Chain Attacks	Poisoned Action and PR injection executed without alerts	Confirms systemic blind spots in workflow provenance checks
Cloud Monitoring	CloudTrail logged events only when enabled; GuardDuty failed on IAM misuse	Detection dependent on log persistence; easily bypassed
Risk Assessment	CVSS scores rated most attacks High (≥7.5) or Critical (≥9.0)	Confirms pipeline abuse represents severe risk to cloud security

7.4 Contributions of the Research

The dissertation contributes to both academic research and industry practice**.**

- Academic Contributions
  It provides reproducible, empirical validation of CI/CD-originated attacks in AWS, addressing a critical gap in the literature where most prior studies focused on theoretical models or post-incident reports rather than controlled experimentation (Beller, Gousios, & Zaidman, 2022; Shahin, Babar, & Zhu, 2017). Furthermore, the application of CVSS v3.1 to pipeline vulnerabilities introduces a structured method for quantifying risk, thereby linking DevSecOps security research with established risk assessment frameworks (FIRST, 2019; NVD, 2024).

- Practical Contributions

  The Five-Pillar DevSecOps Framework developed in Chapter 5 translates experimental findings into actionable security controls. It emphasises the operational importance of log immutability, multi-scanner adoption, and provenance enforcement in pipelines, aligning with industry best practices such as the OWASP CI/CD Security Top 10 (OWASP, 2023) and Google's SLSA framework for supply chain assurance (Google, 2023). By consolidating preventive measures such as least-privilege IAM policies and dependency pinning with detective mechanisms including immutable logging and anomaly monitoring, the framework

provides practitioners with a reproducible and adaptable model for securing CI/CD environments (Wiz, 2023; Sysdig, 2024).

Table 7.2 – Academic vs. Practical Contributions

Contribution Type	Key Contributions	Examples from Study
Academic	Empirical validation of CI/CD abuse	GuardDuty blind to IAM misuse despite CloudTrail logs (Table 5.1)
Academic	Application of CVSS v3.1 to pipeline threats	IAM wildcard and poisoned action scored ≥9.0 Critical
Practical	Five-Pillar DevSecOps Framework	Preventive + detective controls for pipelines (Figure 3.11.3)
Practical	Evidence for log persistence and multi-scanner adoption	CloudTrail disablement blinds GuardDuty; TruffleHog failed, Gitleaks succeeded

7.5 Legal, Social, Ethical, and Professional Reflections

The study adhered to GDPR by using only synthetic secrets and dummy data, avoiding any processing of real user information (European Union, 2018; Information Commissioner's Office [ICO], 2023). All simulated attacks were executed within an isolated AWS Free Tier account, ensuring no third-party systems were impacted. High-risk destructive activities were excluded, keeping the research within ethical hacking principles and AWS's Acceptable Use Policy (Amazon Web Services, 2024).

Professionally, the project modelled responsible security research by anonymising logs and presenting results as guidance for practitioners. This reinforced the researcher's accountability as both a student and an emerging cybersecurity professional, aligning with recognised ethical frameworks such as the ACM Code of Ethics (ACM, 2022) and the BCS Code of Conduct (British Computer Society [BCS], 2023).

7.6 Limitations of the Study

Although the dissertation achieved its objectives, several limitations must be acknowledged to contextualise the findings and assess their generalisability. These limitations are not weaknesses of design but rather reflect the boundaries within which the study was conducted (Sekhar, 2024; Peffers, Rothenberger, & Tuunanen, 2020).

7.6.1 Resource and Technical Constraints

The experiments were limited to AWS Free Tier resources, which restricted the scope of services available and the retention of monitoring logs. This limitation shaped the depth of the analysis, particularly in relation to advanced security services such as Macie or Security Hub that might have provided additional visibility (Amazon Web Services, 2024a). While sufficient for demonstrating pipeline abuse and detection gaps, the restricted environment inevitably reduced the realism of

enterprise-scale deployments, a challenge also noted in prior research where constrained experimental setups traded scalability for reproducibility (Huang, Kumar, & Lewis, 2022; Sekhar, 2024).

7.6.2 Use of Synthetic Credentials and Dummy Data

To ensure GDPR compliance, all secrets and IAM credentials used in the study were synthetically generated (European Union, 2018; Information Commissioner's Office [ICO], 2023). Although this approach replicated the structure of real exposures, it did not trigger the automated incident response mechanisms that AWS applies to genuine compromised keys, such as credential revocation or alerts (Amazon Web Services, 2024). This limitation reduced the ability to fully assess the operational chain of detection and response, though it preserved ethical integrity and maintained compliance with legal and institutional requirements (ACM, 2022).

7.6.3 Scope of Cloud Providers

The study deliberately focused on AWS as a single cloud provider, excluding Microsoft Azure and Google Cloud Platform (GCP). While this focus allowed for a controlled and reproducible environment, it restricts the generalisability of findings to multi-cloud settings where detection tools differ in design and emphasis. For example, Azure's stronger compliance focus through Microsoft Defender for Cloud (Microsoft, 2024) and GCP's data-centric monitoring in Security Command Center (Google Cloud, 2024) were not evaluated, meaning that the results should be interpreted primarily in the context of AWS. This limitation reflects broader observations in cloud security research that provider-specific differences can significantly shape monitoring coverage (CISA & NSA, 2023).

7.6.4 Simplified Pipeline Complexity

The GitHub Actions workflows and Terraform scripts deployed in the study reflected common misconfigurations but did not incorporate the full complexity of enterprise CI/CD pipelines. Features such as container orchestration, multi-stage approvals, and third-party SaaS integrations were excluded to preserve experimental control. As a result, the findings illustrate core vulnerabilities but may under-represent the breadth of attack paths present in larger, production-scale pipelines (Pan, Sun, & Wei, 2024; Ladisa, Mezzina, & Tamburri, 2022).

In summary, these limitations shaped the scope and external validity of the research. They do not undermine the study's contributions but highlight the importance of future work that expands scale, integrates multi-cloud environments, and incorporates real-world operational complexity (ENISA, 2022). By acknowledging these boundaries, the dissertation maintains transparency while ensuring that its findings are appropriately contextualised.

7.7 Future Work

While this dissertation has demonstrated the feasibility of CI/CD pipeline abuse and provided empirical evidence of detection gaps in AWS-native tools, it also highlights opportunities for further investigation. Extending this work would not only address the limitations identified in Section 7.6 but also deepen the practical and academic contributions to pipeline security (Shahin, Babar, & Zhu, 2017; Sekhar, 2024).

7.7.1 Multi-Cloud Investigations

Future research should expand beyond AWS to include Azure DevOps and Google Cloud Platform (GCP). Each provider employs distinct detection strategies—Azure emphasises compliance through Microsoft Defender for Cloud, while GCP's Security Command Center prioritises data exfiltration heuristics (Microsoft, 2024; Google Cloud, 2024). Comparative experiments would allow researchers to determine whether detection gaps are systemic across platforms or provider-specific, offering more generalisable insights (CISA & NSA, 2023).

7.7.2 Enterprise-Scale Pipelines

The pipelines simulated in this study were intentionally simplified for reproducibility. Future experiments should incorporate the complexity of enterprise CI/CD environments, including container orchestration (e.g., Kubernetes), multi-stage approvals, and third-party SaaS integrations. Such setups would expose additional attack surfaces, particularly in supply chain compromise and lateral movement (Pan, Sun, & Wei, 2024; ENISA, 2022; Ladisa, Mezzina, & Tamburri, 2022).

7.7.3 Advanced Supply Chain Threats

While this dissertation tested poisoned GitHub Actions and PR injection, it did not simulate more complex supply chain attacks such as dependency confusion, artefact tampering, or long-term persistence mechanisms. Incorporating these threats in future work would provide a more comprehensive view of pipeline abuse scenarios (Birsan, 2021; Codecov, 2021; Legit Security, 2024; ENISA, 2022).

7.7.4 Integration with Advanced Detection Systems

The reliance on AWS-native monitoring highlighted significant blind spots. Future research should integrate third-party Security Information and Event Management (SIEM) platforms or extended detection and response (XDR) systems to assess whether cross-source correlation improves visibility (CISA & NSA, 2023; Sysdig, 2024). Evaluating the operational overhead of these solutions would also contribute to the practical feasibility of their adoption, as operational complexity and cost have been identified as major barriers to effective cloud monitoring (Legit Security, 2024).

7.7.5 Longitudinal Studies and Behavioral Analysis

This study executed attacks in discrete phases. Future work could extend the analysis by monitoring pipelines over longer periods, incorporating repeated attacks, stealthier persistence techniques, or insider threat simulations. Longitudinal data would provide richer insights into detection timelines and false positive management (Pan, Sun, & Wei, 2024; MITRE, 2024; Sinan, Cavusoglu, & Techatassanasoontorn, 2025).

Table 7.3 – Recommended Future Work Directions

Area	Suggested Focus	Rationale
Multi-cloud research	Azure and GCP alongside AWS	Identify systemic vs. provider-specific detection gaps
Enterprise-scale pipelines	Kubernetes, SaaS integrations, multi-stage approvals	Capture broader attack surface in production-like pipelines
Advanced supply chain threats	Dependency confusion, artefact tampering	Extend scope beyond poisoned actions and PR injection
Detection integration	Third-party SIEM/XDR tools	Test whether external systems improve detection fidelity
Longitudinal studies	Repeated and persistent attack campaigns	Analyse detection timelines and resilience over time

In summary, future research should broaden the scope, deepen the complexity, and extend detection analysis across multiple environments. These directions will strengthen both the academic foundation and practical applicability of CI/CD security research, building on the contributions of this dissertation.

7.8 Closing Statement

This dissertation has demonstrated that CI/CD pipelines, while central to modern software delivery, also represent a critical attack surface capable of enabling infrastructure compromise when misconfigured. Through controlled experimentation using GitHub Actions, Terraform, and AWS Free Tier, the study showed that common missteps such as exposed secrets, unpinned dependencies, and over-permissive IAM policies can be reliably exploited, often without detection by default monitoring tools (Wiz, 2023; Sysdig, 2024).

By empirically validating these risks and translating the findings into a reproducible Five-Pillar DevSecOps Framework, the research has made contributions of both academic and practical value. It has confirmed gaps highlighted in prior literature, extended them with reproducible evidence, and provided industry practitioners with actionable strategies to strengthen pipelines against real-world abuse (OWASP, 2023; Google, 2023).

Ultimately, securing CI/CD pipelines is not merely a technical challenge but a professional responsibility. The study reinforces the principle that security must be embedded into automation from the outset, with layered, tamper-resistant controls forming the foundation of resilient DevSecOps practice (CISA & NSA, 2023). While limitations remain, the work provides a foundation for future research into multi-cloud and enterprise-scale environments, contributing to the ongoing effort to safeguard software supply chains in an era of accelerating cloud adoption (ENISA, 2022).

Bibliography

- Amazon Web Services (AWS). (2024a). *Amazon GuardDuty documentation.* https://docs.aws.amazon.com/guardduty/
- Amazon Web Services (AWS). (2024b). *Security best practices in IAM.* https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
- Beller, M., Gousios, G., Zaidman, A., & Juergens, E. (2022). When CI/CD goes wrong: Developer trade-offs in pipeline security. *Empirical Software Engineering, 27*(5), 1–29. https://doi.org/10.1007/s10664-022-10112-5
- Birsan, A. (2021). Dependency confusion: Exploiting the dependency supply chain. *Medium.* https://medium.com/@alex.birsan/dependency-confusion-4a5d60f2da41
- Bondalapati, A., & Malaraju, K. (2025). Design patterns in secure CI/CD pipeline design. *IEEE Software Engineering Journal, 38*(2), 63–78.
- Bridgecrew. (2023). *Checkov documentation.* https://www.checkov.io/
- Castro, M. (2024). GuardDuty anomaly detection effectiveness in CI/CD contexts. *IEEE Security & Privacy, 22*(1), 55–62.
- Center for Internet Security (CIS). (2023). *CIS Terraform Benchmark v1.1.0.* https://www.cisecurity.org
- CISA. (2023). *Lessons learned from the SolarWinds and Codecov supply chain compromises.* Cybersecurity and Infrastructure Security Agency. https://www.cisa.gov
- CISA & NSA. (2023). *Securing CI/CD pipelines guidance.* https://www.cisa.gov/resources-tools/resources/securing-cicd-pipelines
- Cloud Native Computing Foundation (CNCF). (2022). *GitOps principles.* https://opengitops.dev
- Codecov. (2021). *Codecov Bash Uploader Security Update.* https://about.codecov.io/security-update/
- Dasanayake, M., Rajapaksha, A., & Perera, A. (2023). Agile-based threat simulation for cloud-native CI/CD pipelines. *ACM Transactions on Software Engineering and Methodology, 32*(1), 1–27.
- Ehrman, J. (2025). CI/CD pipelines as infrastructure: Threat modelling and mitigation. *Journal of Cyber Engineering, 9*(1), 67–82.
- European Union Agency for Cybersecurity (ENISA). (2022). *Threat landscape for supply chain attacks.* https://www.enisa.europa.eu/publications/threat-landscape-for-supply-chain-attacks
- García, L. (2025). Pull request injection: Weaponization of GitHub workflows. *Xygeni Research.* https://xygeni.io/blog
- GitGuardian. (2024). *State of Secrets Sprawl Report 2024.* https://www.gitguardian.com/state-of-secrets-sprawl-report-2024
- GitHub. (2023a). *GitHub Actions documentation.* https://docs.github.com/en/actions
- GitHub. (2023b). *The State of the Octoverse.* https://octoverse.github.com
- GitLab. (2023). *Global DevSecOps Survey.* https://about.gitlab.com/devsecops-survey/
- Google. (2023). *Supply chain Levels for Software Artifacts (SLSA).* https://slsa.dev
- Google Cloud. (2024). *Security Command Center overview.* https://cloud.google.com/security-command-center

- HashiCorp. (2023). *Terraform documentation.* https://developer.hashicorp.com/terraform/docs
- Huang, R., Kumar, A., & Lewis, J. (2022). Runtime misconfiguration risk in Terraform deployments. *IEEE Cloud Computing, 10*(2), 44–53.
- Ladisa, A., Mezzina, C. A., & Tamburri, D. A. (2022). A taxonomy of software supply chain attacks. *ACM Computing Surveys, 55*(7), 1–38. https://doi.org/10.1145/3546066
- Legit Security. (2023). *CI/CD threat report.* https://www.legitsecurity.com/research
- Legit Security. (2024). *State of CI/CD security practices 2024.* https://www.legitsecurity.com
- Marandi, M., Chatterjee, A., & Iyer, L. (2023). Detecting policy drift in cloud IAM configurations. *International Journal of Information Security Science, 12*(3), 203–217.
- Microsoft. (2024a). *Microsoft Defender for Cloud documentation.* https://learn.microsoft.com/en-us/defender-for-cloud/
- Microsoft Security Response Center. (2023). *Threats targeting build pipelines.* https://msrc.microsoft.com/blog
- MITRE. (2024). *ATT&CK for Cloud Matrix (including T1554 Supply Chain Compromise).* https://attack.mitre.org
- Morrison, P., Bellomo, S., & Jones, J. (2015). Infrastructure as code: A systematic literature review. *ACM SIGSOFT Software Engineering Notes, 40*(5), 1–8.
- National Institute of Standards and Technology (NIST). (2022). *Secure Software Development Framework (SSDF), SP 800-218.* https://csrc.nist.gov/publications/detail/sp/800218/final
- National Vulnerability Database (NVD). (2024). *Common Vulnerability Scoring System v3.1 Calculator.* https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator
- OWASP. (2023a). *CI/CD Security Cheat Sheet.* https://cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet.html
- OWASP. (2023b). *OWASP Top 10 for CI/CD Security.* https://owasp.org/www-project-cicd-security-top-10/
- Palo Alto Networks – Unit 42. (2024). *2024 Cloud Threat Report.* https://unit42.paloaltonetworks.com/cloud-threat-report-2024/
- Pan, Y., Sun, J., & Wei, X. (2024). Insecure automation: A large-scale analysis of CI/CD workflows. In *Proceedings of the 45th IEEE Symposium on Security and Privacy* (pp. 113–127).
- Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2020). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS07421222240302
- Pressman, R. S. (2010). *Software engineering: A practitioner's approach* (7th ed.). McGraw-Hill.
- Rajapakse, D., Gousios, G., & Zaidman, A. (2021). CI/CD tool usage and adoption across large-scale projects. *Empirical Software Engineering, 26*(5), 87–103.
- Roper, K. (2024). Terraform security: IAM permissions in practice. *DevSecOps Journal, 12*(1), 33–49.
- Sekhar, A. (2023). Agile methodologies for managing uncertainty in technical research. *International Journal of Agile Systems, 5*(2), 88–102.

- Sekhar, M. (2024). Experimental frameworks for pipeline security research. *Journal of Secure Computing, 15*(2), 115–131.*
- Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review. *IEEE Access, 5,* 3909–3943.
- Sinan, B., Cavusoglu, H., & Techatassanasoontorn, A. (2025). The adoption of policy-as-code in SME cloud pipelines. *Journal of Cybersecurity Practice and Research, 13*(1), 21–36.*
- Sinha, A., Gupta, A., & Singh, R. (2022). Secret scanning in DevOps pipelines: Tools, techniques, and limitations. *International Journal of Information Security, 21*(3), 239–256.* https://doi.org/10.1007/s10207-021-005573
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
- Sysdig. (2024). *Cloud-Native Threat Report 2024.* https://sysdig.com/reports/cloud-native-threat-report-2024/
- Tenable. (2024). *Terrascan by Tenable documentation.* https://www.tenable.com/solutions/terrascan
- The Hacker News. (2025). *CVE-2025-30066: GitHub Action hijack exposes secrets.* https://thehackernews.com/2025/03/github-action-exploit.html
- Trend Micro. (2024). *Cloud misconfiguration threats report.* https://www.trendmicro.com/vinfo/us/security/news/security-technology/cloud-misconfiguration
- TruffleHog. (2024). *Secrets scanning documentation.* https://github.com/trufflesecurity/trufflehog
- Wiz. (2023). *Software supply chain threats: CI/CD exploitation and trust boundary failures.* Wiz Research. https://www.wiz.io/blog/cicd-attacks
- Xcitium. (n.d.). *Top 5 AWS misconfigurations and how to detect them.* https://www.xcitium.com/aws-misconfigurations/
- Xygeni. (2025). *CI/CD pipeline pull request threat analysis.* https://xygeni.io